

Факультет компьютерных наук | Машинное обучение | Москва 2026

Лекция 1
Введение в машинное обучение

Преподаватель: Меликян Алиса Валерьевна, amelikyan@hse.ru
кандидат наук, доцент Департамента программной инженерии ФКН НИУ ВШЭ

О курсе

Цель курса: изучение ключевых понятий, методов и моделей классического машинного обучения, а также развитие практических навыков их использования.

В результате будут сформированы компетенции по применению машинного обучения для проведения исследований и решения прикладных задач.

Оценивание

- Практические задания – 30%
- Контрольная работа 1 – 20%
- Контрольная работа 2 – 20%
- Итоговый проект – 30%

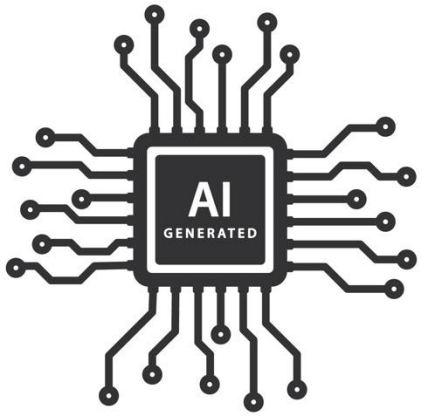
Квизы

Регулярно на занятии проводятся квизы по пройденным материалам. За них можно получить дополнительные баллы, которые добавляются к оценкам за практические задания. Это необязательная форма контроля, пропуск квиза не влияет на оценки.

- Первое место (с учетом правильности и скорости ответов) – 2 балла.
- Второе и третье места – 1 балл.
- Верно выполнено более 80% заданий – 0,5 баллов.

Оценки 9 и 10

В случае правильного выполнения задания выставляется оценка **8 баллов**. Для получения более высокой оценки нужно выйти за рамки задания, применить дополнительные приемы исследования данных, которые позволят получить новые знания об изучаемых объектах или выявить интересные тенденции в рассматриваемых данных. По каждому заданию будут даны рекомендации в каком направлении можно двигаться для получения дополнительных ценных результатов в рамках изучаемой темы.



Использование ИИ

Использование ИИ разрешено, но должно быть задекларировано в соответствии с правилами НИУ ВШЭ.

Студент (или группа студентов) обязаны сопроводить письменное задание специальным разделом «Описание применения генеративной модели», где обозначить:

- части текста, подготовленные с использованием генеративных моделей (вся работа, отдельные разделы или отдельные фрагменты работы);

- для каждой такой части текста – цели и способ применения генеративных моделей (полностью сгенерированный текст, сгенерированный текст, подвергнутый авторской редакции или авторский текст с использованием сгенерированных материалов);

- для каждой использованной генеративной модели – ее название, адреса сайта в интернете (либо описания иного источника модели);

- оценку эффективности применения генеративных моделей для достижения поставленных целей.

Где взять данные?



- Готовые датасеты
- Ручной сбор (опросы, интервью, фото, видео, сбор из интернета)
- Веб-скрейпинг, API
- Краудсорсинг (Яндекс Задания)
- Синтетические данные

Источники данных для анализа (российские)

- RLMS-HSE
- Данные НИУ ВШЭ
- Росстат
- Портал открытых данных правительства Москвы
- Если быть точным

Источники данных для анализа (иностранные)

- Kaggle
- Google Dataset Search
- Harvard Dataverse
- Awesome data
- Eurobarometer
- World Bank Open Data

Веб-скрейпинг



Веб-скрейпинг

Веб-скрейпинг – это автоматизированный сбор данных с веб-страниц. Можно использовать когда нужной информации нет в удобном виде (нет API или готового файла с данными).

В процессе сбора данных можно столкнуться с ограничениями:

- страницы сайта могут иметь разную структуру или меняться (нужно адаптировать скрейпер);
- частые запросы могут создавать нагрузку на сервер и вести к блокировкам;
- некоторые сайты используют защиту от ботов (CAPTCHA, блокировки IP) и др.

Но, для всего этого есть решения!

Этапы веб-скрейпинга

1. Анализ сайта и выбор данных
Определяем с какого сайта и какие данные нужно собрать. Если есть API, то используем его. Если нет, то изучаем структуру страниц (HTML теги).

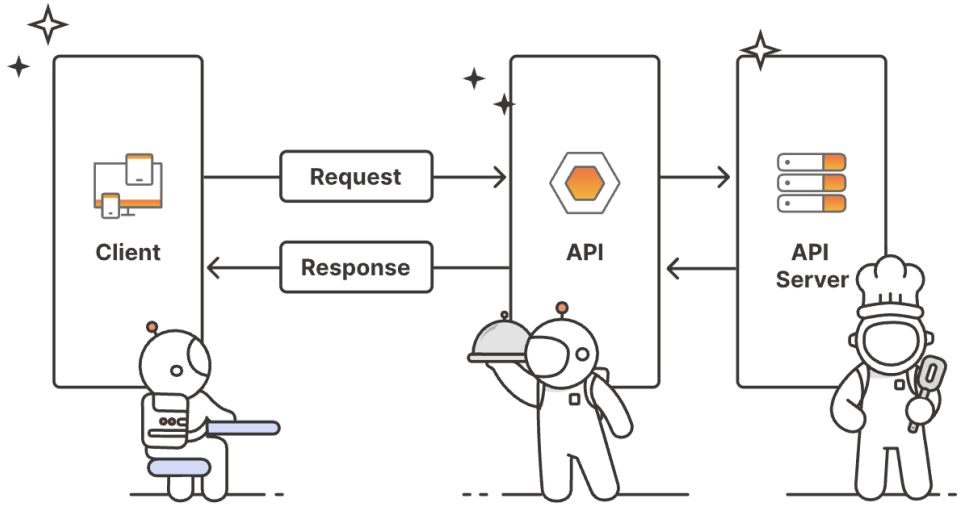
2. Фетчинг (загрузка страниц)
Отправляем HTTP-запросы к нужным URL. Получаем HTML-код страниц или данные в других форматах (JSON, XML). Часто используется библиотека requests.

3. Парсинг содержимого
Разбираем полученный HTML/JSON, чтобы извлечь нужные элементы. Выделяем нужные данные по тегам или классам, можем искать по атрибутам тегов. Часто используется библиотека BeautifulSoup.

4. Краулинг (при необходимости)
Если нужно собрать данные со многих страниц, то переходим по ссылкам и повторяем процесс.

5. Обработка и очистка и сохранение данных

API



API (Application Programming Interface) – это интерфейс для взаимодействия программ и «официальный» способ получить данные с соблюдением правил сайта.

Представляет собой набор правил и методов, с помощью которых одна программа может «общаться» с другой, например, запрашивать данные.

Мы отправляем запрос с нужными параметрами, который сервер обрабатывает и возвращает данные в формате JSON или XML.

Пример API Википедии

```
<?xml version="1.0"?>
<api batchcomplete="">
  <continue cmcontinue="20250617154017|78147827" continue="-||" />
  <query>
    <categorymembers>
      <cm pageid="81547074" ns="0" title="SDSS J120136.02+300305.5" />
      <cm pageid="81309696" ns="14" title="Category:Physical horizons" />
      <cm pageid="77326503" ns="0" title="Negative air ions" />
      <cm pageid="80064998" ns="0" title="Frenesy (physics)" />
      <cm pageid="78751748" ns="0" title="Missile lofting" />
      <cm pageid="79966709" ns="0" title="Haloscope (physics)" />
      <cm pageid="1310583" ns="14" title="Category:History of physics" />
      <cm pageid="925607" ns="14" title="Category:Physics education" />
      <cm pageid="79026168" ns="0" title="Wohlfarth Lectureship" />
      <cm pageid="78245824" ns="0" title="Nottingham effect" />
    </categorymembers>
  </query>
</api>
```

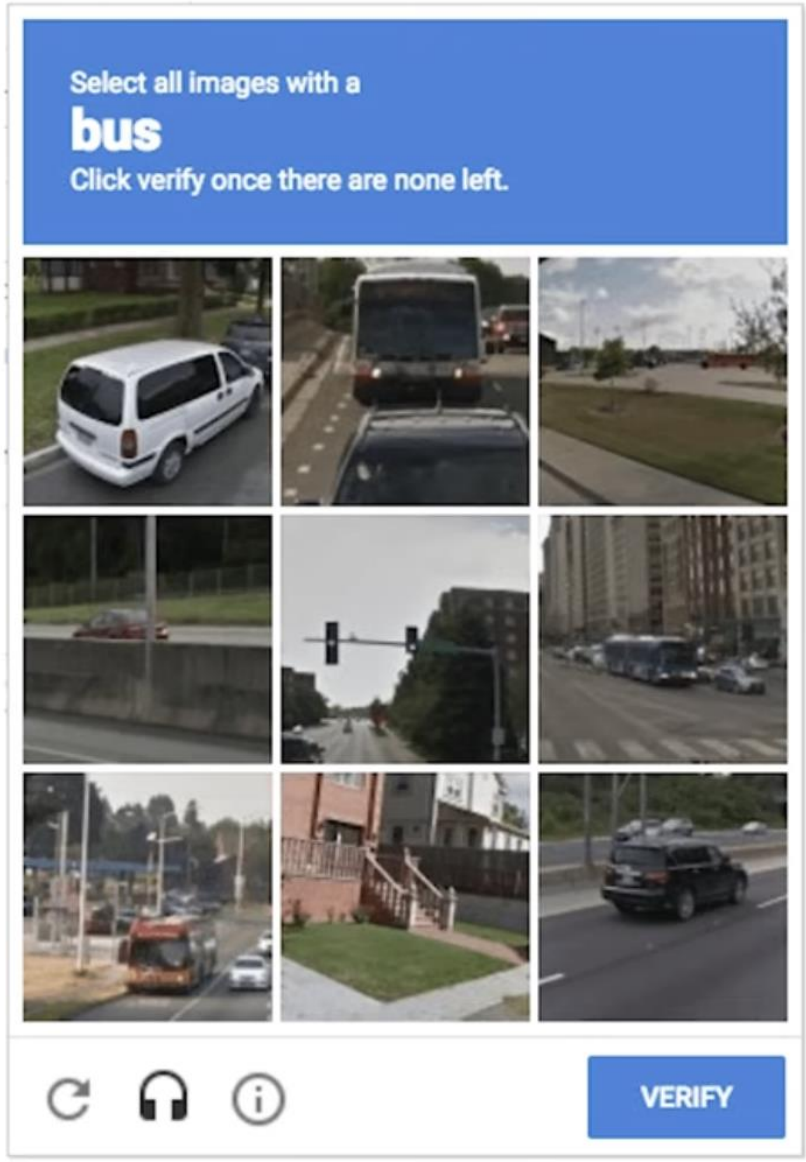
https://en.wikipedia.org/w/api.php?action=query&list=categorymembers&cmtitle=Category:Physics&cmsort=timestamp&cmdir=desc&format=xmlfm

Строка https://en.wikipedia.org/w/api.php (до знака вопроса) – это точка входа в API. Всё, что идёт после знака вопроса – это, собственно, запрос. Он представляет собой что-то вроде словаря и состоит из пар «ключ=значение», разделяемых амперсандом &. Запрос вернет список подкатегорий, относящихся к категории **Physics** (физика), отсортированных по времени обновления в формате XML.

Краудсорсинг данных

Краудсорсинг данных предполагает привлечение сообщества (обычно через онлайн-платформы) для решения задачи по сбору или обработке данных. Например, для сбора и разметки изображений, текстов или аудиозаписей для обучения моделей. Работа часто оплачивается, но может быть и волонтерской. В России для этого можно использовать Яндекс Задания.

На картинке пример того как вы бесплатно размечаете изображения для Google, возможно, сами того не зная.



Синтетические данные

Синтетические данные – это искусственно созданные данные, которые алгоритмы генерируют так, чтобы они напоминали реальные по структуре и статистическим закономерностям. Их используют для обучения и тестирования моделей, когда доступ к настоящим данным ограничен или невозможен. Такие данные помогают воспроизвести общие паттерны, но не содержат информации о реальных событиях или людях. Могут не учитывать все нюансы и сложные зависимости, наблюдаемые в реальности.

Принципы генерации данных

1. **Данные соответствуют предметной области:** учитывают ограничения реальных процессов, не содержат невозможные или противоречивые значения.
2. **Сохранены статистические свойства признаков:** сохраняются реальные распределения значений признаков, воспроизведены меры центральной тенденции и разброса, а также важнейшие корреляции. Можно привести доказательства в виде сравнения описательных статистик, графиков распределений и корреляционных матриц.
3. **Данные обладают вариативностью и разнообразием значений**.
4. **Алгоритм генерации прозрачен и воспроизводим, указаны его ограничения**.

Когда можно использовать синтетические данные в исследованиях и КР/ВКР?

- Объективно нет данных, необходимых для решения задач исследования. Нужно это обосновать. Нежелание тратить время и силы на сбор данных не является объективной причиной использования синтетических данных.
- Цель работы – не исследовать объективную реальность, а, например, разработать алгоритм или методику, и протестировать их на разных данных.
- Для демонстрации работы прототипа цифрового продукта.
- Более безопасный вариант – использовать синтетические данные вместе с реальными. Например для аугментации данных (Data Augmentation), когда существующий набор данных искусственно увеличивается и повышается разнообразие данных без изменения их смысла.

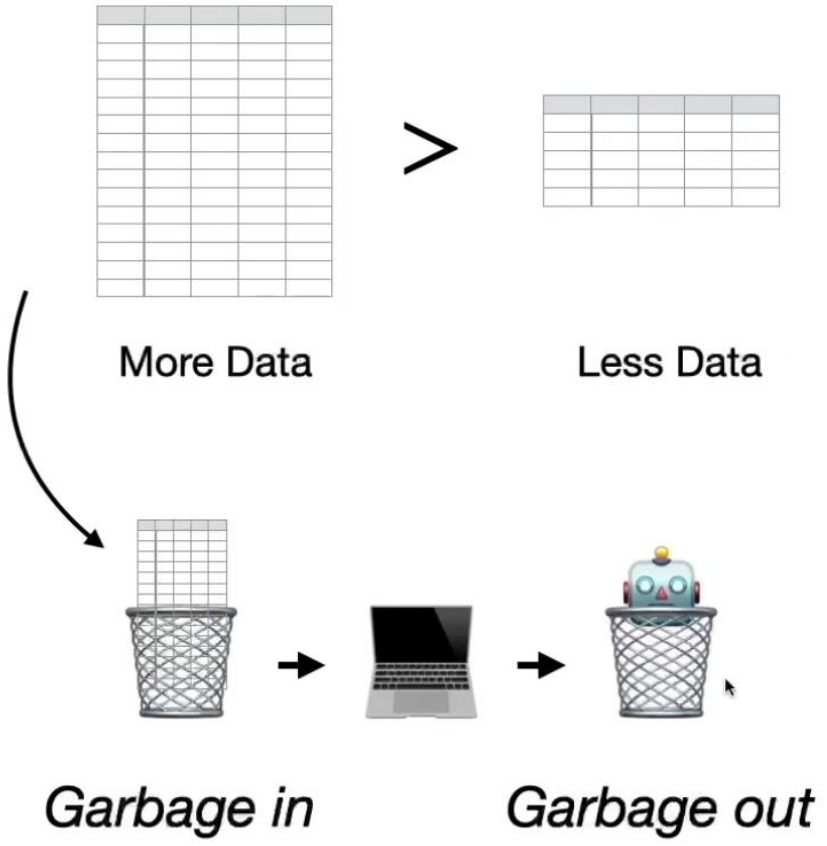
Когда не рекомендуется использовать синтетические данные в исследованиях и КР/ВКР?

- Результаты исследования претендуют на практическую ценность для отрасли.
- В исследовании делаются обобщающие выводы о реальном мире.
- Сгенерированные данные являются единственным основанием для выводов исследования.
- Не обосновано сходство с реальными данными на основе статистического сравнения или экспертной оценки.

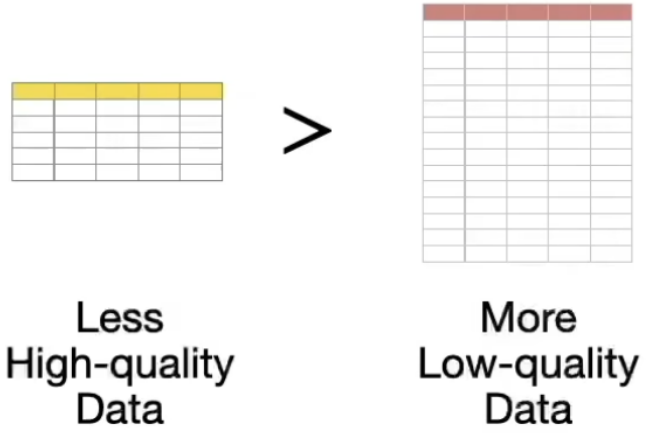
Отчет об использовании синтетических данных

- В разделе «Ограничения исследования» необходимо прямо указать об использовании в работе синтетических данных и отметить, что это накладывает ограничения на интерпретацию полученных результатов, в связи с чем выводы не могут быть напрямую перенесены на реальные данные без дополнительной валидации.
- Необходимо указать причину использования синтетических данных, детально описать принципы генерации и обосновать их.
- Требуется детальное описание используемого метода генерации (модель, параметры, и пр.).
- Привести доказательство сходства с реальными данными (если применимо).
- Привести воспроизводимый программный код, который использовался при генерации данных.

Чем больше данных, тем лучше?



Ответ: Не всегда. Данные должны быть точными (корректно оценивать реальность), а выборка должна быть репрезентативной.

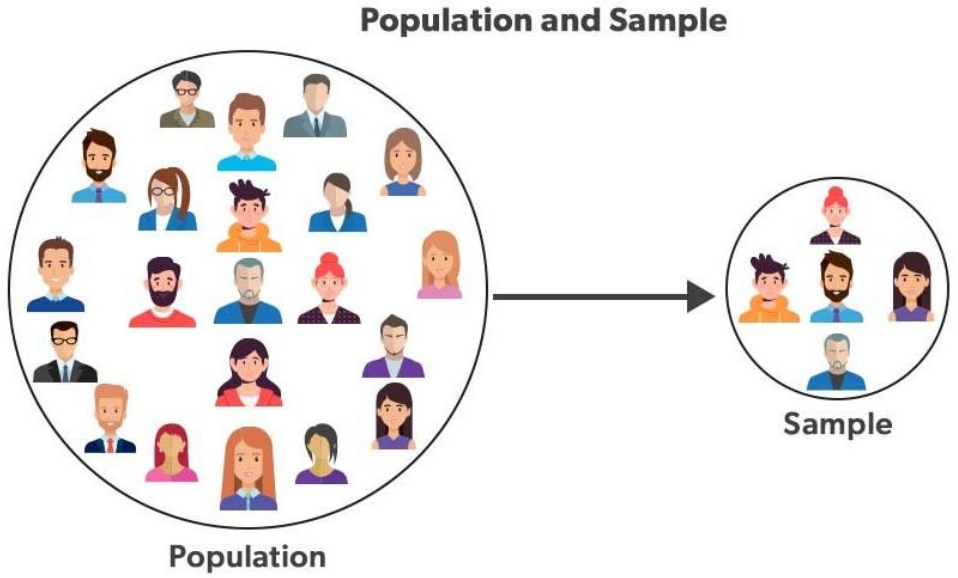


Выборка и генеральная совокупность

Генеральная совокупность (Population) – это совокупность всех объектов, для которых модель должна работать корректно (все сотрудники компании, все возможные транзакции). В большинстве случаев крайне сложно получить доступ к данным по всей генеральной совокупности.

Выборка (Sample) – это часть генеральной совокупности, которая нам доступна и на которой мы обучаем и оцениваем модель. Важно, чтобы выборки были репрезентативными.

Основная цель ML: обучить модель на выборке так, чтобы она хорошо работала на генеральной совокупности. Это называется обобщающая способность (generalization).



Подходы к формированию выборки

1. Простая случайная выборка (Simple Random Sampling)

Каждый элемент имеет равную вероятность быть выбранным, при этом выбор одного элемента не влияет на выбор других.

2. Систематическая выборка (Systematic Sampling)

Предполагает отбор каждого n-го элемента из генеральной совокупности. Например, если у вас есть список студентов и вы выбираете каждого 10-го студента.

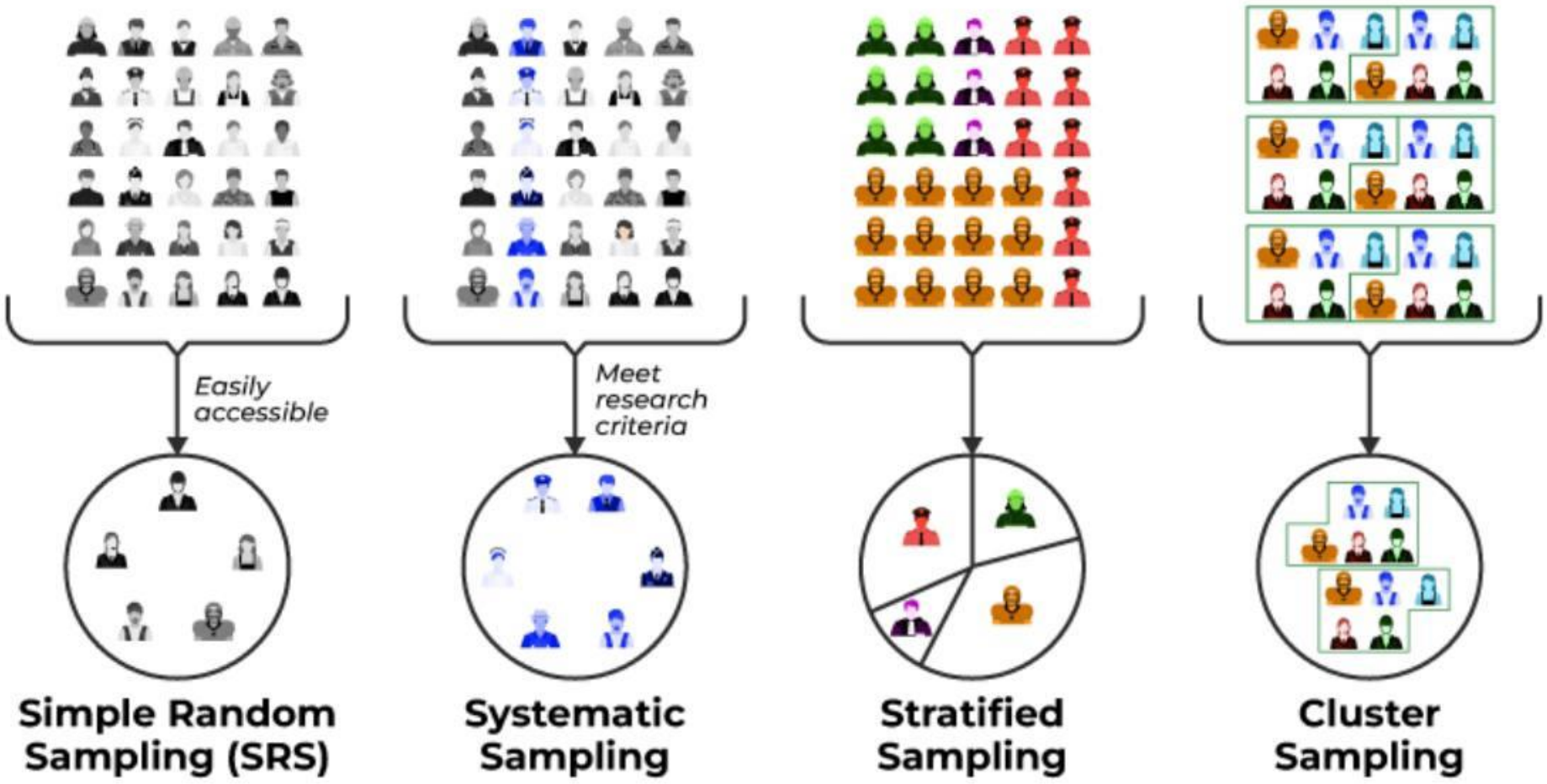
3. Стратифицированная выборка (Stratified Sampling)

Генеральная совокупность делится на подгруппы (страты) по определенным характеристикам (например, возраст, пол), а затем из каждой страты случайным образом отбираются элементы. Этот метод обеспечивает репрезентативность каждой подгруппы.

4. Кластерная выборка (Cluster Sampling)

Генеральная совокупность делится на кластеры или группы, часто по географическому принципу. Случайным образом отбираются кластеры, и затем все элементы внутри выбранных кластеров включаются в выборку.

Подходы к формированию выборки

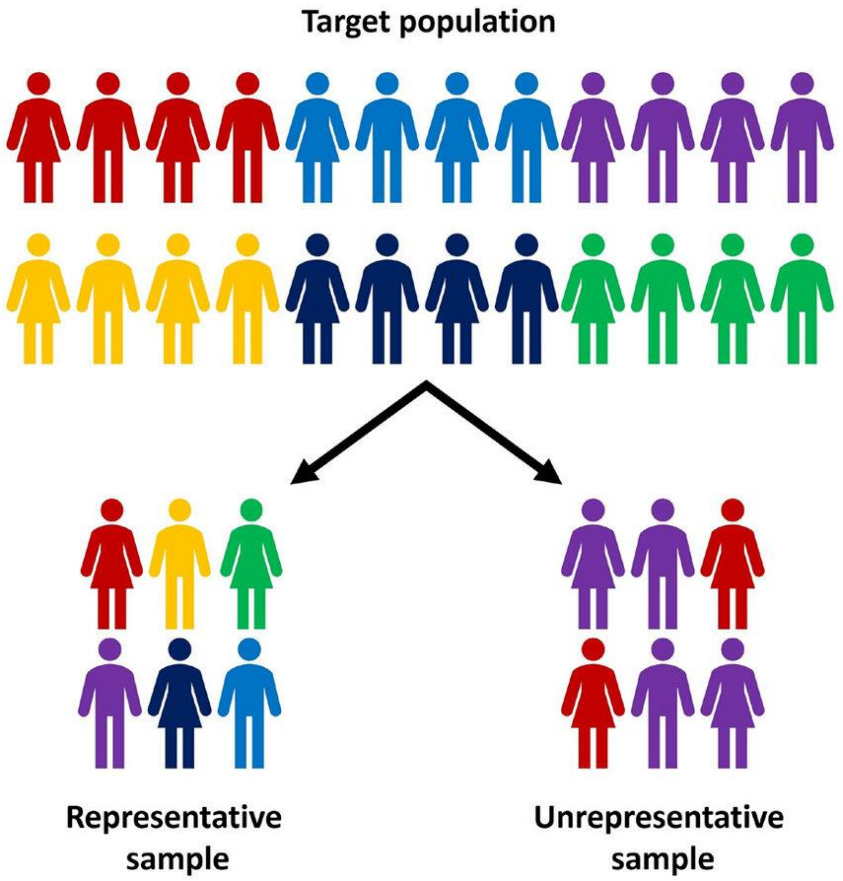


Репрезентативность выборки

Репрезентативность выборки (Sample Representativeness) – это свойство выборки корректно отражать генеральную совокупность, чтобы модель, обученная на этих данных, хорошо обобщалась на реальные (production) данные.

Выборка репрезентативна, если:

- распределения признаков в выборке похожи на распределения в продакшене;
- присутствуют все важные подгруппы объектов в близкой пропорции;
- нет систематических перекосов (bias).



Репрезентативность выборки: пример из ПИ

Задача: предсказать какие задачи в проекте будут просрочены.

Нерепрезентативная выборка:
- только задачи одного проекта;
- только задачи junior-разработчиков;
- данные только за период перед релизом;
- только крупные задачи.

Репрезентативная выборка:
- разные проекты и команды;
- разные уровни разработчиков;
- разные типы задач;
- разные периоды разработки;
- и выполненные, и просроченные задачи.

Машинное обучение

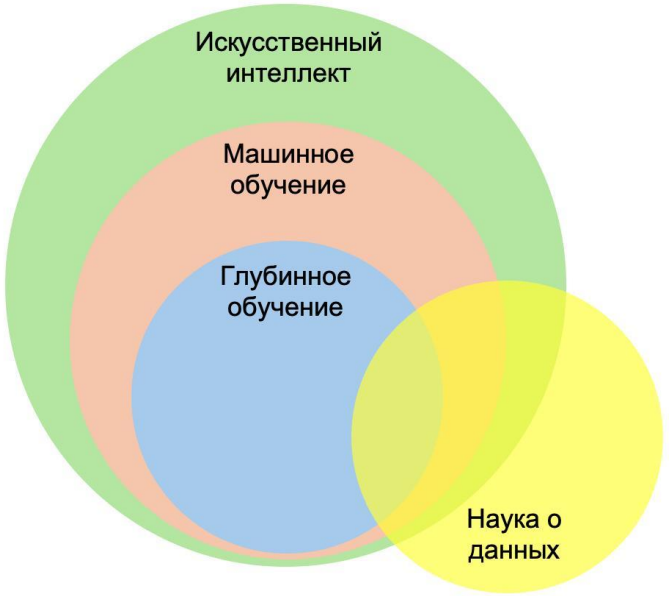
Машинное обучение (Machine Learning, ML) – изучает методы построения моделей, способных обучаться на основе имеющихся данных. Лежит в основе множества технологий: рекомендательные системы в интернет-магазинах, интеллектуальные фильтры спама, системы распознавания речи и лиц, системы медицинской диагностики и т.д.

Суть машинного обучения в том, чтобы научить компьютер автоматически выявлять закономерности в данных. Это осуществляется путем построения математических моделей, которые обучаются на основе входных данных и используют полученные знания для анализа новых данных и принятия решений.

Например, если нужно прогнозировать спрос на товары, то мы даем модели данные, модель «обучается» на них и далее строит свои прогнозы, на основе выявленных закономерностей.

Области знаний

- **Искусственный интеллект (Artificial Intelligence)** – область компьютерных наук, занимающаяся созданием систем, способных выполнять задачи, требующие интеллектуальных способностей.
- **Машинное обучение (Machine Learning)** – набор методов, позволяющих строить модели, которые обучаются на данных, выявляют закономерности и обобщают их на новые данные.
- **Глубинное обучение** (**Deep Learning**) – подмножество методов ML, основанных на глубоких нейронных сетях.
- **Наука о данных (Data Science)** – объединяет всё, что относится к работе с данными (ML + базы данных + визуализация + статистика + бизнес-аналитика + …). Ее цель – превратить данные в решение для бизнеса или науки. Может вообще не использовать AI.



Классическое МО или нейронные сети?

Классическое МО используем когда объекты обладают разнородными осмысленными признаками и когда важна интерпретируемость, например, в медицине, финансах, госуправлении.
Примеры: прогноз цены квартиры, кредитный скоринг, кластеризация клиентов

Нейронные сети сами извлекают важные признаки из данных. Используются когда признаки объекта однородные и не несут смысла (пиксели на изображении, буквы в тексте).
Примеры: определение объекта на фото, классификация отзывов по тону, распознавание речи, генерация изображений.

Примеры задач машинного обучения

- **Классификация** – определение категории, к которой относится объект. Например, является ли письмо спамом или нет.
- **Регрессия** – предсказание численного значения. Например, на какую заработную плату может рассчитывать программный инженер с определенным опытом работы, набором компетенций, профессиональных и личных качеств.
- **Кластеризация** – поиск структур или групп в данных. Например, сгруппировать программные проекты по их характеристикам (размеру, сложности, количеству ошибок, используемым технологиям), чтобы для каждой группы выбрать подходящий подход к разработке и тестированию.

Примеры применения МО в ПИ

- Прогноз качества кода (Code Quality Prediction) – прогноз качества кода на основе его структуры, истории изменений и предыдущих дефектов.
- Прогноз производительности разработчика/команды (Developer Productivity Prediction) – прогноз будущей производительности разработчика или команды на основе показателей их работы.
- Классификация дефектов (Defect Classification) – определение типа и категории обнаруженной ошибки: логическая, синтаксическая, производительности, безопасности и т. д.

Классическое Обучение

Данные заранее категоризированы или численные

С учителем

Предсказать категорию

Классификация

«Разложи носки по цвету»



Предсказать значение

Регрессия

«Разложи галстуки по длине»



Данные никак не размечены

Без учителя

Разделить по схожести

Кластеризация

«Разложи похожие вещи по кучкам»



Найти зависимости

Уменьшение Размерности (обобщение)

«Собери из вещей лучшие наряды»



Выявить последовательности

Ассоциация

«Найди какие вещи я часто ношу вместе»



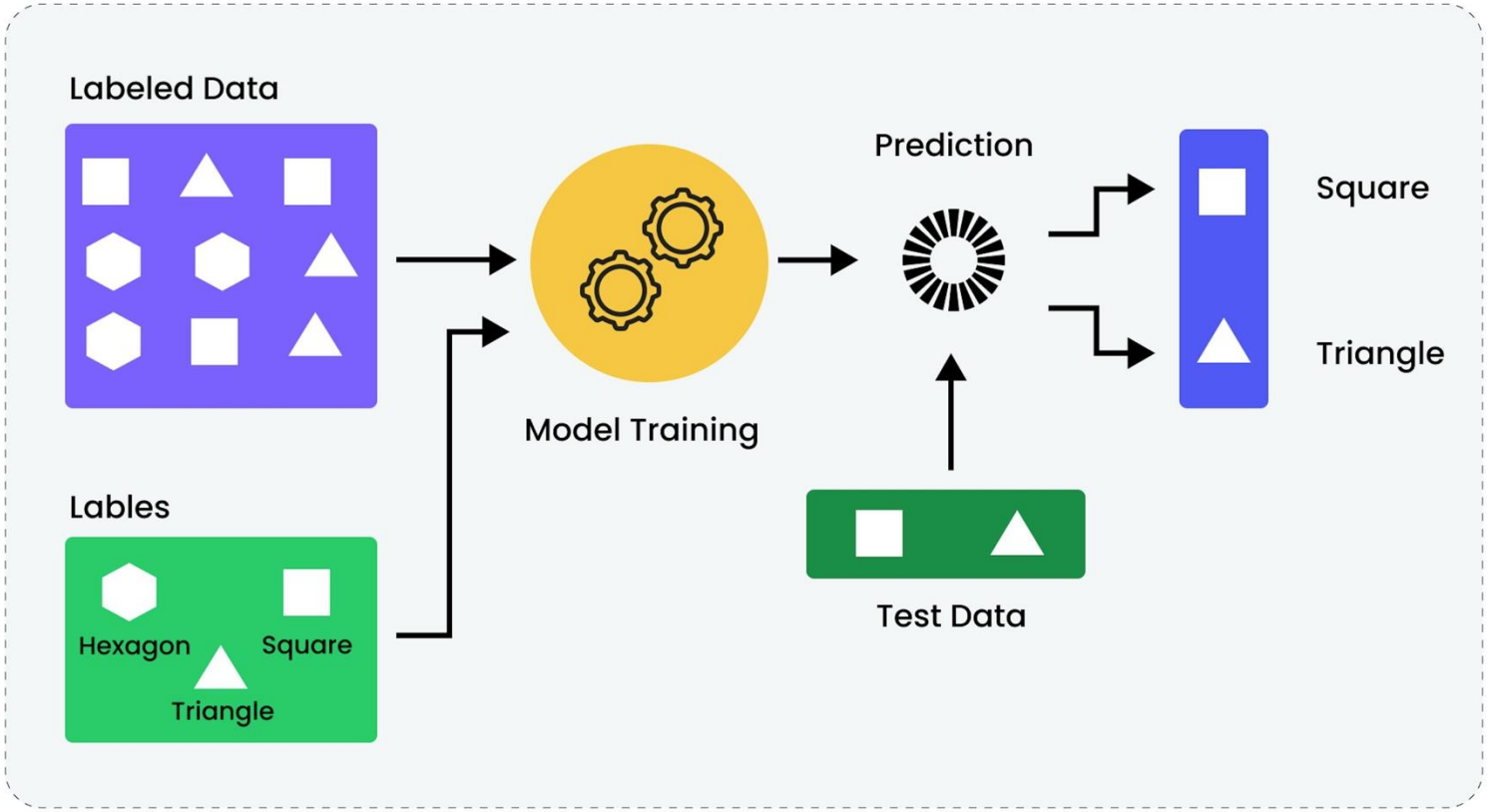
Алгоритмы машинного обучения

- **Только для решения задач классификации**: Наивный байесовский классификатор, Логистическая регрессия.
- **Только для решения задач регрессии**: Линейная регрессия (классическая и полиномиальная), Гребневая регрессия (Ridge), Регрессия Лассо (Lasso), Регрессия ElasticNet.
- **Для решения как задач классификации, так и регрессии**: Алгоритм k-ближайших соседей (k-NN), Метод опорных векторов (SVM), Деревья решений, Случайный лес (Random Forest), Градиентный бустинг.

Обучение с учителем

Примеры задач: классификация, регрессия.

Цель – научиться делать точные предсказания для новых, ранее не встречавшихся данных.

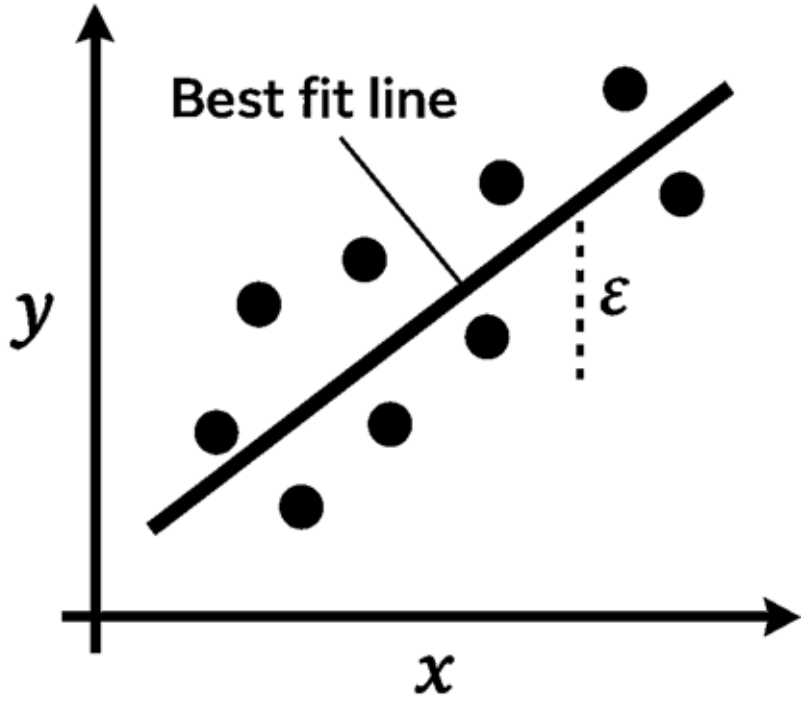


Алгоритмы для обучения с учителем

Линейный алгоритм

- ⊕ Достаточно простой и интерпретируемый алгоритм;
- ⊕ Подходит для малых объёмов данных;
- ⊕ Каждый коэффициент несет смысловую нагрузку;
- ⊕ Теоретически хорошо изучен.

- ⊖ Восстанавливает лишь линейные закономерности;
- ⊖ Дает не очень точные предсказания;
- ⊖ На сложных данных не получить хорошее качество.

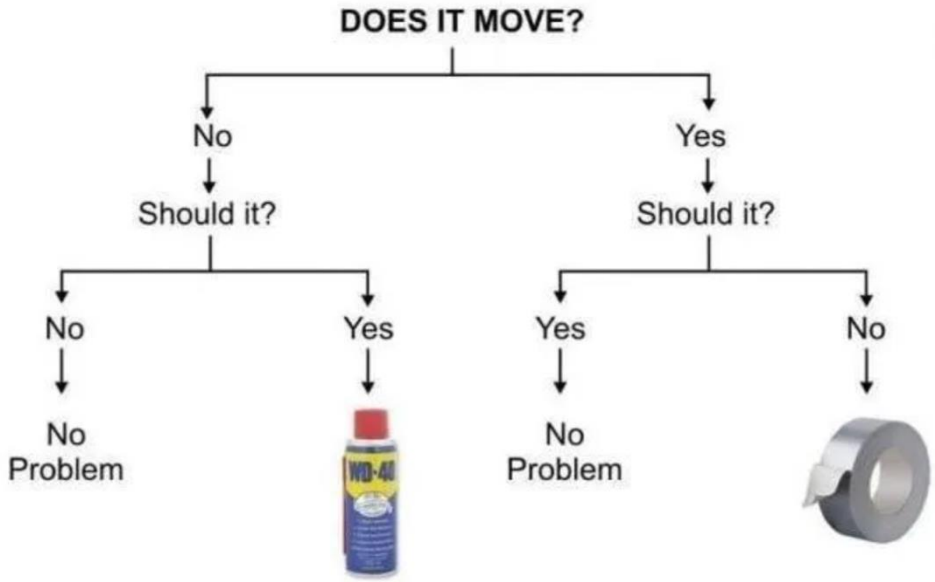


Алгоритмы для обучения с учителем

Деревья решений

- ⊕ Простота и интерпретируемость;
- ⊕ Не требуют масштабирования признаков;
- ⊕ Умеют выявлять и моделировать сложные нелинейные связи.

- ⊖ Могут легко переобучаться на тренировочных данных, особенно если дерево глубокое и содержит много узлов.
- ⊖ Небольшие изменения в данных могут привести к построению совершенно другого дерева.

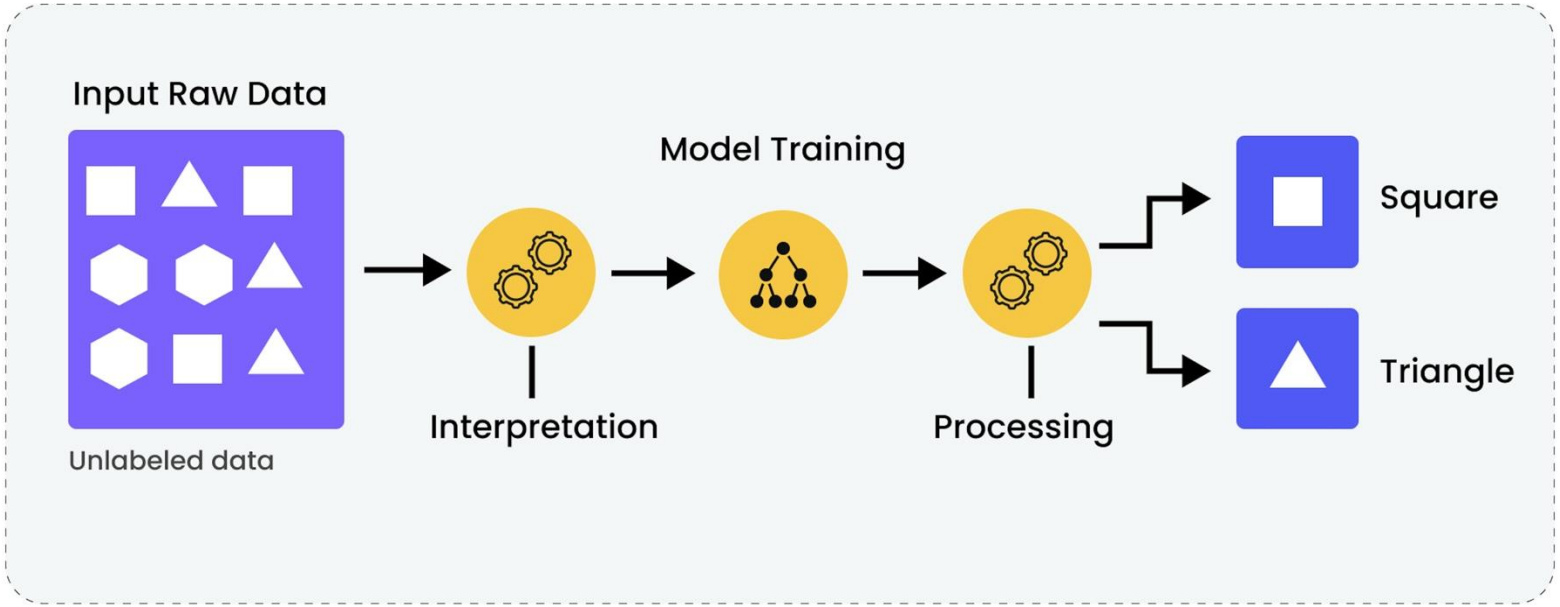


Обучение без учителя

Примеры задач: кластеризация; снижение размерности; поиск аномалий; выявление скрытых структур.

Цель – выявление скрытой структуры, закономерностей и зависимостей в данных.

Нет «правильных ответов», поэтому оценка качества сложнее.



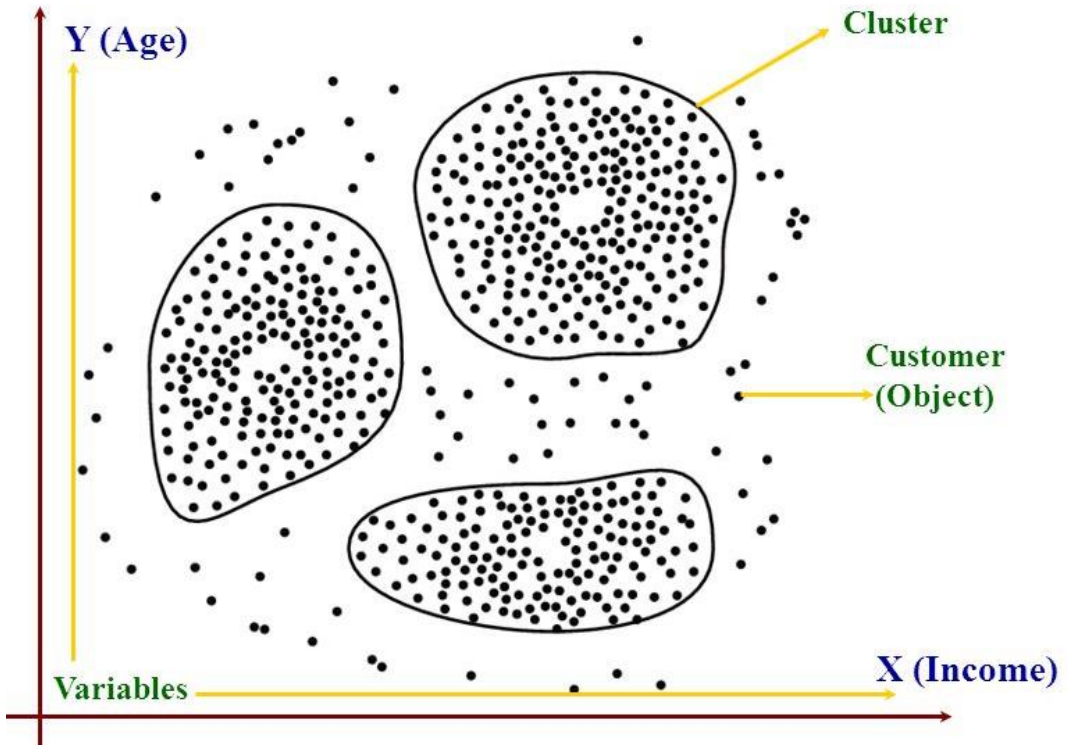
Алгоритмы для обучения без учителя

Кластерный анализ

Группирует объекты так, чтобы элементы внутри одного кластера были похожи друг на друга, а кластеры были достаточно различны между собой.

Возможные цели:

- Выявить скрытую структуру в данных;
- Разделить объекты на однородные группы без заранее заданных меток;
- Упростить дальнейший анализ больших массивов данных.



Алгоритмы для обучения без учителя

Кластерный анализ

- ⊕ Не требует размеченных данных;
- ⊕ Много алгоритмов для разных ситуаций;
- ⊕ Может работать с большими наборами данных.

- ⊖ Сложно выбрать оптимальное число кластеров;
- ⊖ Результаты могут отличаться в зависимости от выбранного алгоритма и его параметров;
- ⊖ Нет универсального критерия качества кластерной модели;
- ⊖ Некоторые алгоритмы чувствительны к шуму и выбросам;
- ⊖ Результаты могут быть трудны для интерпретации.

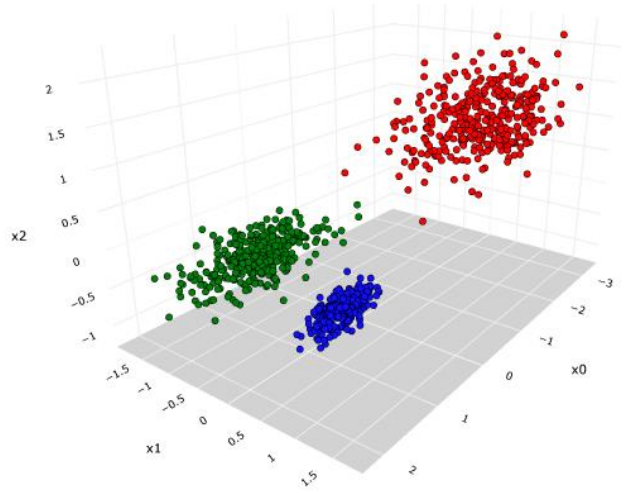
Алгоритмы для обучения без учителя

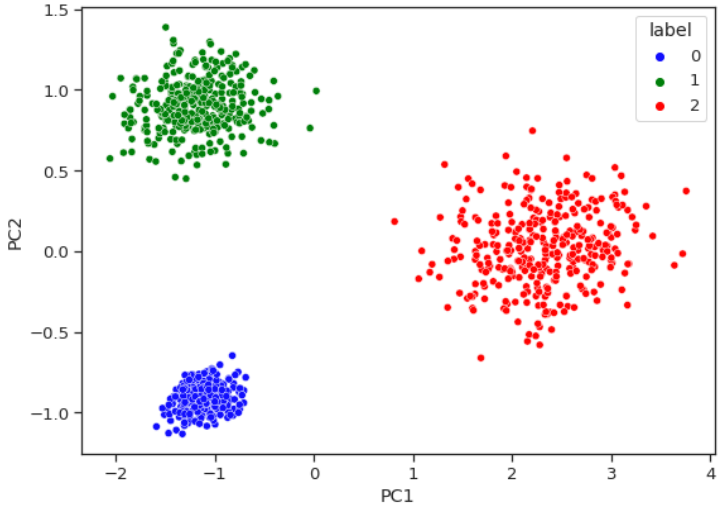
Снижение размерности

Позволяют уменьшить число признаков без существенной потери информации.

Может потребоваться, чтобы упростить анализ и визуализацию данных, ускорить обучение моделей машинного обучения, уменьшить переобучение, устранить избыточные и коррелированные признаки.

Нередко используется перед построением моделей кластеризации, регрессии или классификации.





Алгоритмы для обучения без учителя

Снижение размерности

- ⊕ Ускорение вычислений;
- ⊕ Улучшение обобщающей способности моделей;
- ⊕ Устранение мультиколлинеарности;
- ⊕ Повышение интерпретируемости;
- ⊕ Возможность визуализации сложных данных.

- ⊖ Потеря части информации;
- ⊖ Сложность интерпретации новых признаков;
- ⊖ Не всегда улучшает качество модели;
- ⊖ Результаты зависят от выбранного метода.

Пример: задача скоринга

На основе характеристик клиента (пол, возраст, доход, кредитная история, семейное положение, место работы и пр.) мы хотим предсказать вернет ли он кредит или нет. В данном случае мы решаем задачу бинарной классификации.

Целевая переменная (target) – это величина, которую нужно предсказать. В данном случае это число (1 – если вернет кредит, 0 – иначе). В зависимости от типа целевой переменной и решаемой задачи мы подбираем подходящую модель машинного обучения.

Признаки (features) – характеристики объекта, в данном случае клиента, такие как пол, возраст, доход.

Объекты (objects) – сущности, с которыми мы работаем в рамках решения задачи, в данном случае клиенты.

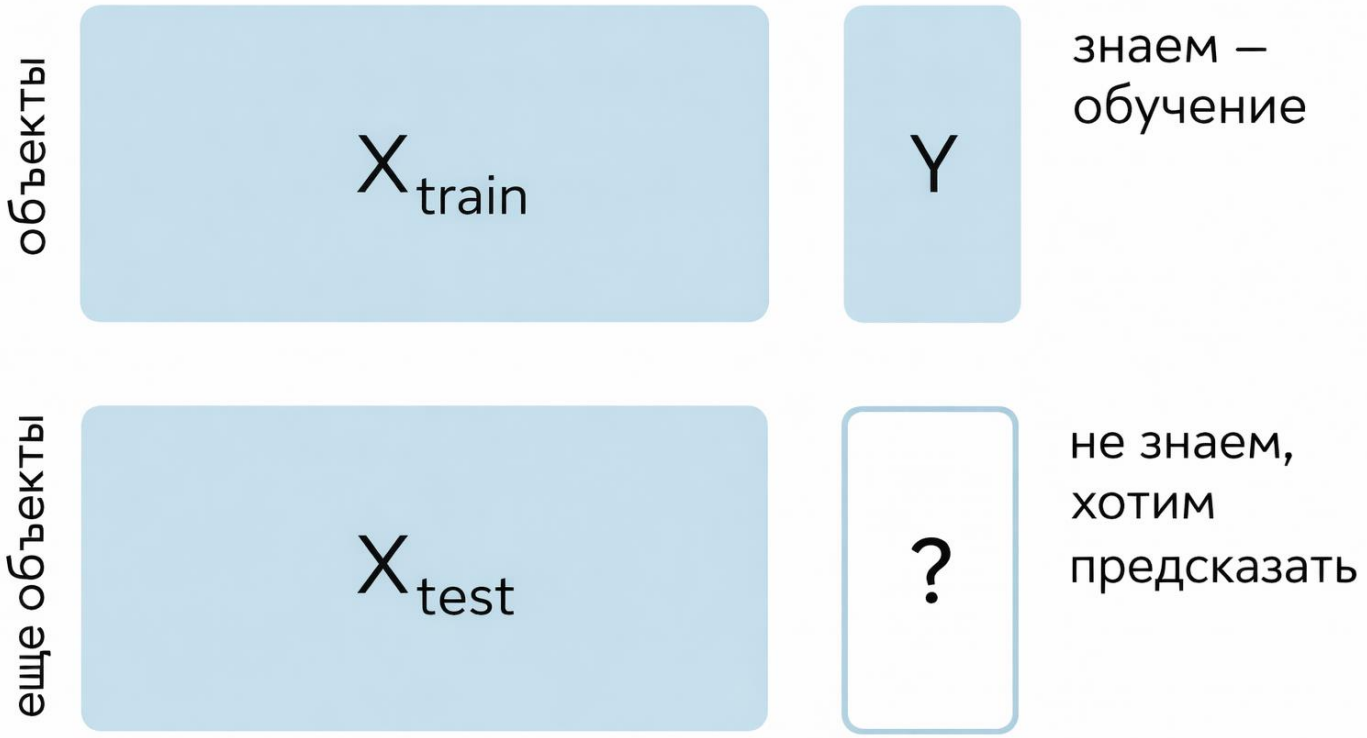
Признаки и целевая переменная

Case	Attributes				Decision
	Length	Height	Width	Weight	Quality
1	4.7	1.8	1.7	1.7	high
2	4.5	1.4	1.8	0.9	high
3	4.7	1.8	1.9	1.3	high
4	4.5	1.8	1.7	1.3	medium
5	4.3	1.6	1.9	1.7	medium
6	4.3	1.4	1.7	0.9	low
7	4.5	1.6	1.9	0.9	very-low
8	4.5	1.4	1.8	1.3	very-low

Целевая переменная (target)

Признаки (атрибуты, фичи, features)

Обучающая и тестовая выборки



Обучающая и тестовая выборки

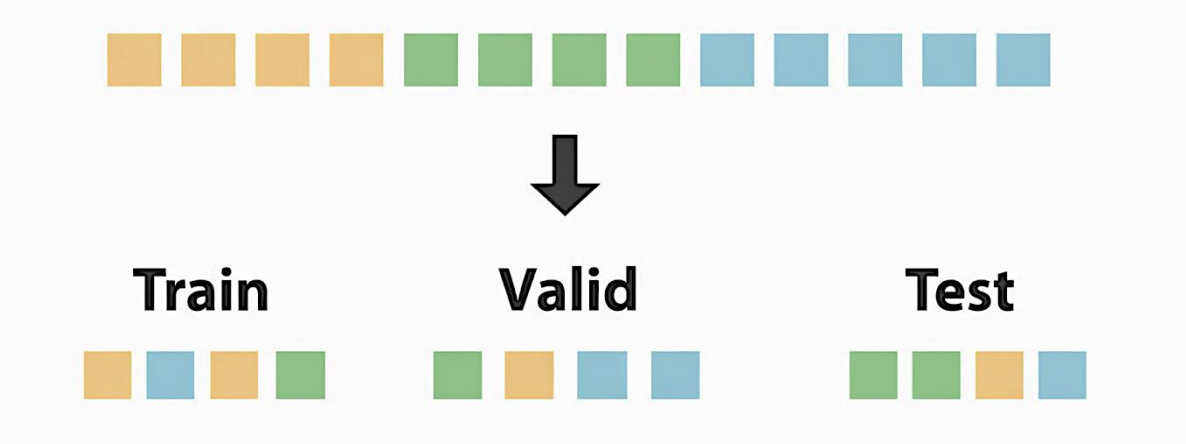
Обучающая выборка (Train Sample) – это часть исходных данных, которая используется для обучения модели машинного обучения, то есть для подбора её параметров и исследования особенностей взаимосвязей между признаками и целевой переменной. Обычно составляет 60–80% всего датасета.

Тестовая выборка (Test Sample) – это часть данных, не участвующая в обучении, но используемая для оценки качества модели. Используется для проверки способности модели обобщать новые данные. Обычно составляет 20–40% датасета.



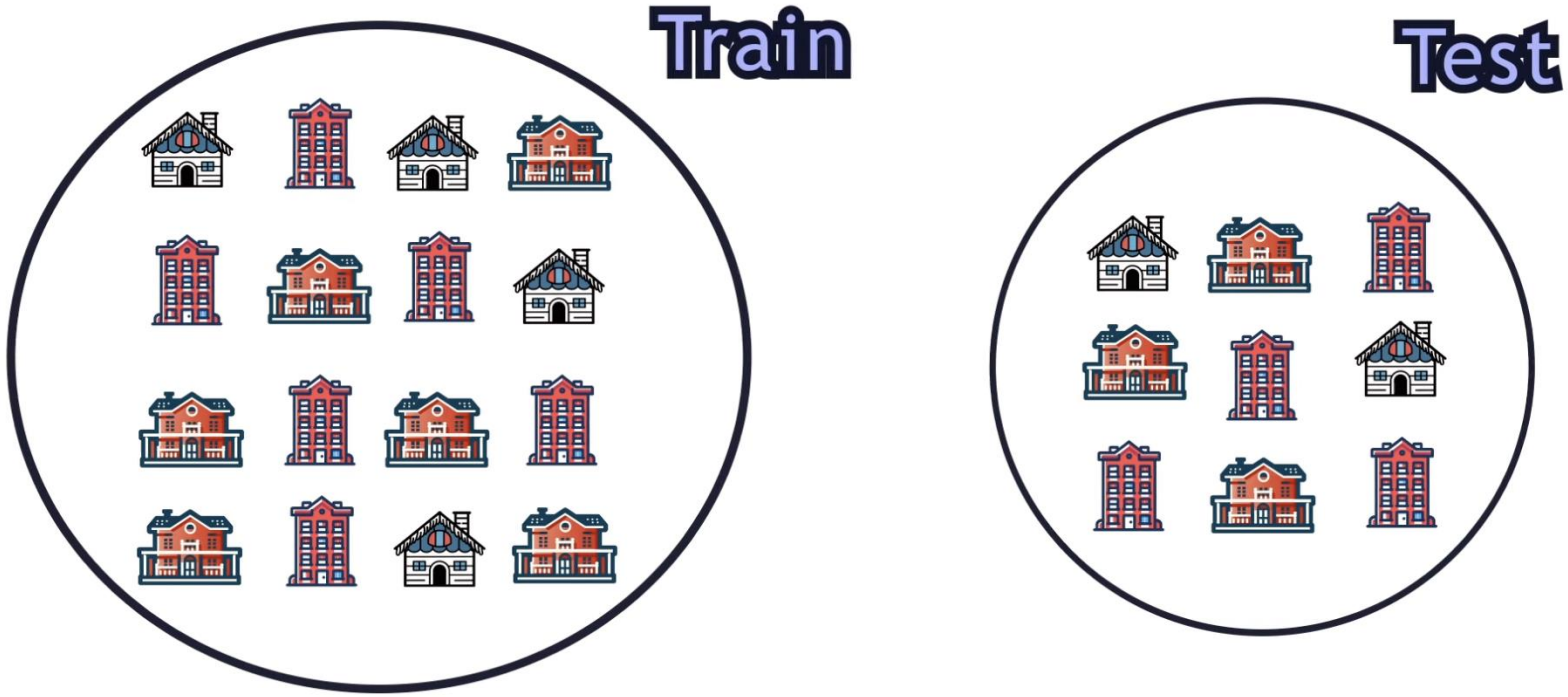
Перемешивание данных

Перед разбиением на train-test может потребоваться перемешать данные (shuffle). Это нужно, чтобы избежать смещения, если строки в датасете изначально как-то упорядочены, например по времени или классу. Это также позволит обеспечить репрезентативность подвыборок. Перемешивать данные не надо, если их порядок несёт информацию, которая для нас важна при моделировании, например, если мы работаем с временными рядами.



Обучающая и тестовая выборки

Обе выборки должны быть репрезентативны.



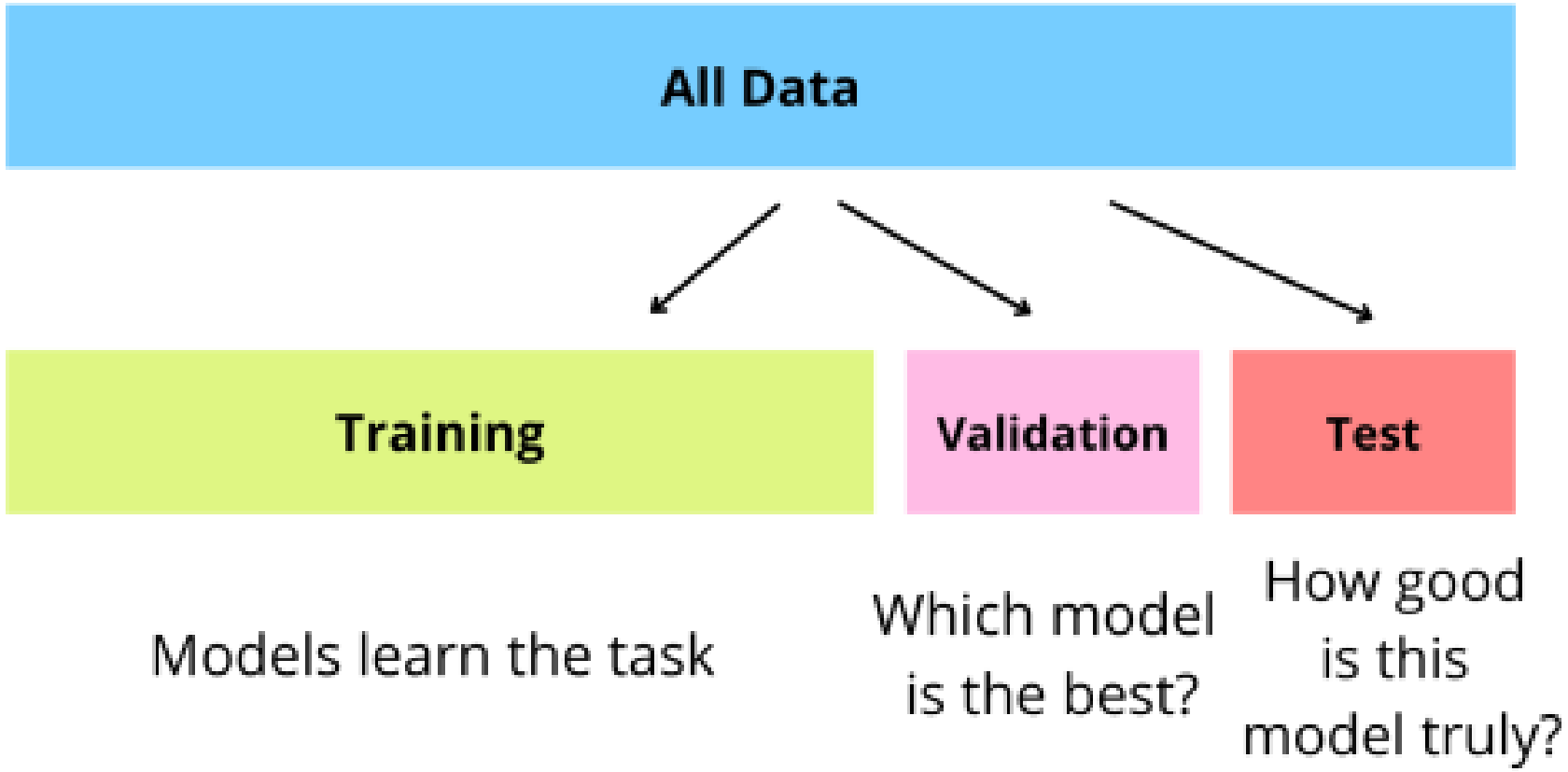
Валидационная выборка

Валидационная выборка (Validation Sample) – это часть данных, используемая для настройки или выбора модели, определения значений гиперпараметров, но не для обучения и не для финальной оценки качества модели.

В случае ее использования данные делятся на три части:

- Обучающая выборка – обучение модели (около 60%);
- Валидационная выборка – настройка и выбор модели (около 20%);
- Тестовая выборка – финальная оценка качества модели (около 20%).

Train – Validation – Test



Кросс-валидация

Кросс-валидация (k-fold cross-validation) – метод оценки качества модели машинного обучения.

- Исходная выборка разбивается на k непересекающихся подвыборок (фолдов) приблизительно одинакового размера. На каждом шаге k−1 фолдов используются для обучения модели, а оставшийся фолд – для её тестирования (валидации).
- Процедура повторяется k раз. Итоговая оценка качества модели определяется как среднее значение метрик, полученных на всех итерациях.
- Все этапы предобработки данных (нормализация, кодирование, отбор признаков) должны выполняться внутри каждого фолда, чтобы избежать утечки данных.

Кросс-валидация



Кросс-валидация

Преимущества:

- Погрешность оценки уменьшается, т.к. используется весь набор данных.
- Выше вероятность подобрать оптимальные гиперпараметры алгоритма.
- Особенно эффективно при ограниченном объёме данных, где важно максимально использовать доступную информацию.

Ограничения:

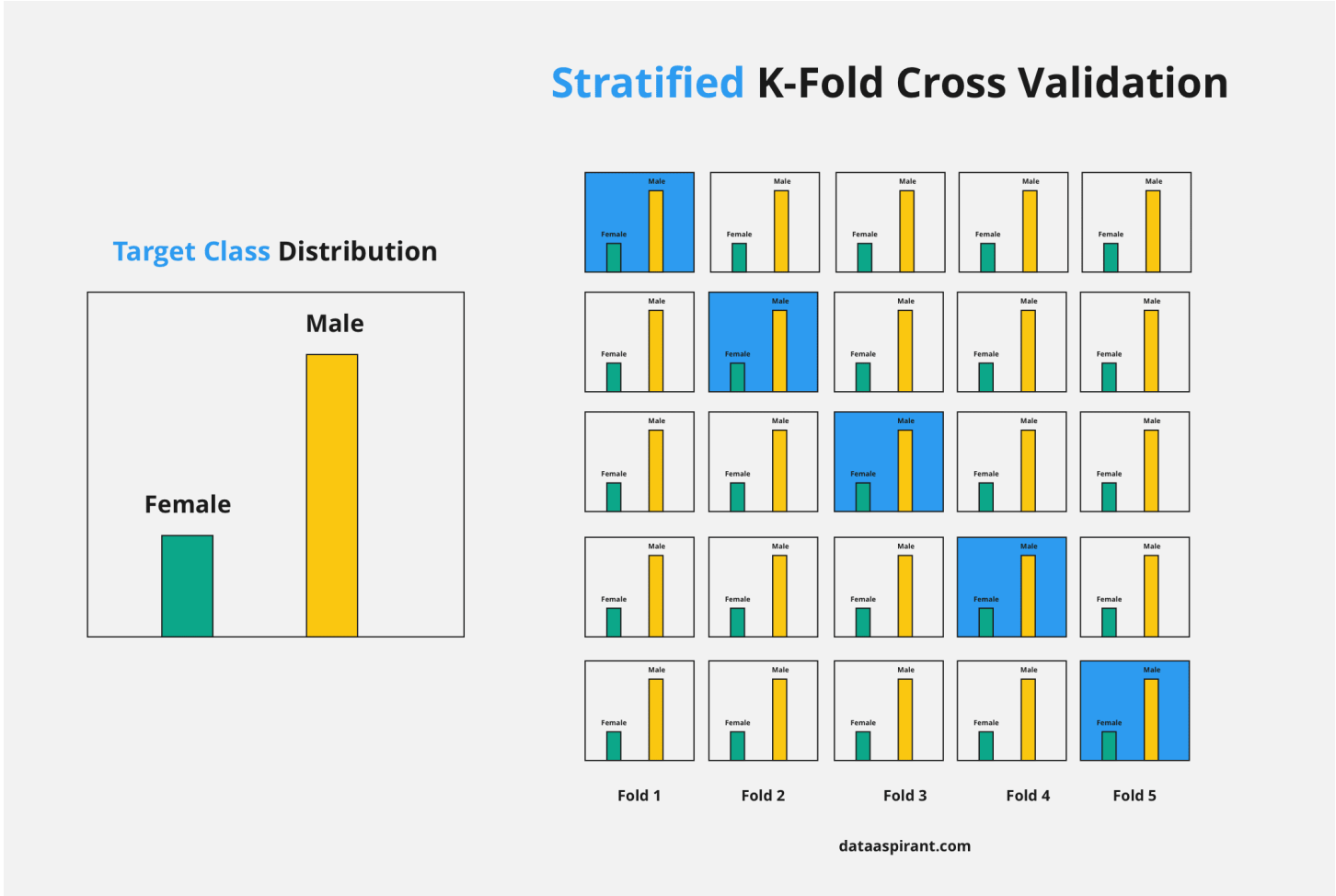
- Обучение производится k раз. Для некоторых моделей это может быть долго.
- При несбалансированных классах стандартная кросс-валидация может давать искажённые оценки, поэтому требуется использование стратифицированной кросс-валидации.

Стратифицированная кросс-валидация

Стратифицированная k-fold кросс-валидация – это модификация классической кросс-валидации для задач классификации, в которых классы представлены несбалансированно.

- Исходная выборка также разбивается на *k* фолдов, но с сохранением пропорций классов. К каждом фолде доля объектов каждого класса приблизительно соответствует их доле во всей выборке.
- Далее процесс обучения и оценки модели полностью аналогичен стандартной k-fold кросс-валидации
- В основном применима к задачам классификации, для регрессии используется редко (возможна стратификация по бинам целевой переменной). При крайне малом числе объектов редкого класса даже стратификация не всегда гарантирует наличие достаточного количества примеров в каждом фолде.

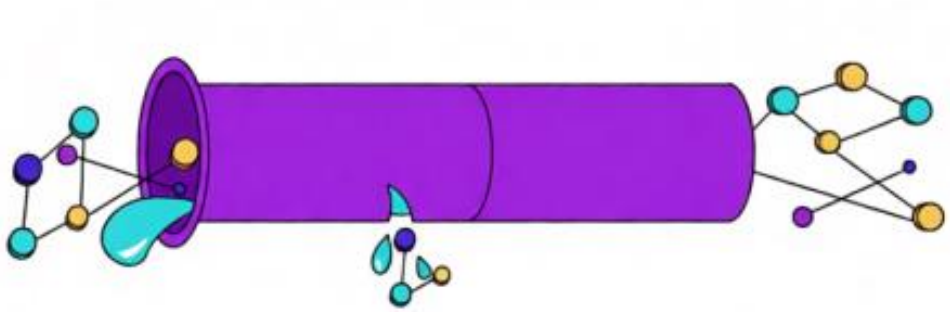
Стратифицированная кросс-валидация



Утечка данных

Утечка данных (Data Leakage) в ML – это ситуация, когда в модель попадает информация, которой в ней быть не должно, и по факту не будет при ее применении в проде. Из-за этого метрики качества модели на train и validation могут стать завышенными, а в проде модель будет плохо работать. Основные виды утечек:

1. Утечка из будущего (Temporal Leakage)
2. Утечка через таргет (Target Leakage)
3. Утечка при препроцессинге
4. Утечка через train/test split



Утечка из будущего (Temporal Leakage)

Модель во время обучения или оценки получает информацию из будущего, которая в реальности будет недоступна в момент предсказания.

Пример: при построении модели, которая предсказывает, успешно ли разработчик справится с задачами в новом проекте, в обучающей выборке используется информация о количестве закрытых им задач и качестве кода за первые месяцы работы. Эти данные появляются только после того, как разработчик уже начал работать в проекте. Модель может получиться очень точной на исторических данных, но на практике не будет работать: в момент, когда нужно сделать предсказание для нового разработчика, этих данных ещё нет.

Как избежать: проверять мог ли признак быть известен в момент предсказания.

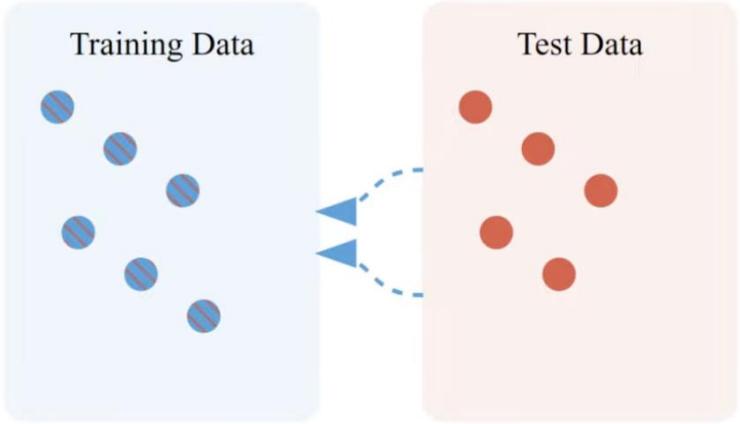
Утечка через таргет (Target Leakage)

Признаки напрямую или косвенно содержат информацию о целевой переменной.

Пример: в задаче оценки производительности сотрудников модель использует в качестве признаков показатели, вычисленные на основе итоговых оценок эффективности (например, средний балл по итогам года), которые фактически являются частью целевой переменной. Модель получает информацию о самом результате и переобучается, что делает прогнозы нереалистично точными, но бесполезными в реальной работе.

Как избежать: анализировать бизнес-логику генерации признаков, проверять корреляции и подозрительно сильные признаки.

Утечка при препроцессинге



Данные из тестовой или валидационной выборки используются на этапе подготовки данных, например, при масштабировании или нормализации признаков по всему датасету целиком. Модель получает косвенные сведения о тестовых данных уже на этапе обучения, что искажает оценку её качества.

Пример: при нормировании значений зарплаты используется среднее арифметическое и стандартное отклонение по всему набору данных, включая тестовую выборку. Это даёт модели доступ к информации о распределении данных, которые в реальной ситуации ей недоступны.

Как избежать: выполнять препроцессинг (нормализация, заполнение пропусков, PCA) на train отдельно, использовать Pipeline / ColumnTransformer

Утечка через train/test split

Возникает, когда обучающая и тестовая выборки оказываются не полностью независимыми друг от друга. Может произойти при случайном разбиении данных когда связанные наблюдения (например, записи об одном и том же сотруднике за разные периоды) попадают и в train, и в test. Модель запоминает специфические паттерны конкретных объектов, что приводит к завышенной оценке качества и иллюзии хорошей обобщающей способности.

Как избежать: применять временной или group-based split и контролировать пересечения сущностей.



Сигналы о возможной утечке данных

- Экстремально высокие метрики качества модели на обучающей выборке.
- Резкое падение качества в проде.
- Один-два признака имеют очень высокую важность.
- Слишком стабильные результаты на кросс-валидации.

Обучение модели

Обучение модели – это процесс поиска оптимальных значений параметров (или весов), при которых минимизируется функция потерь.



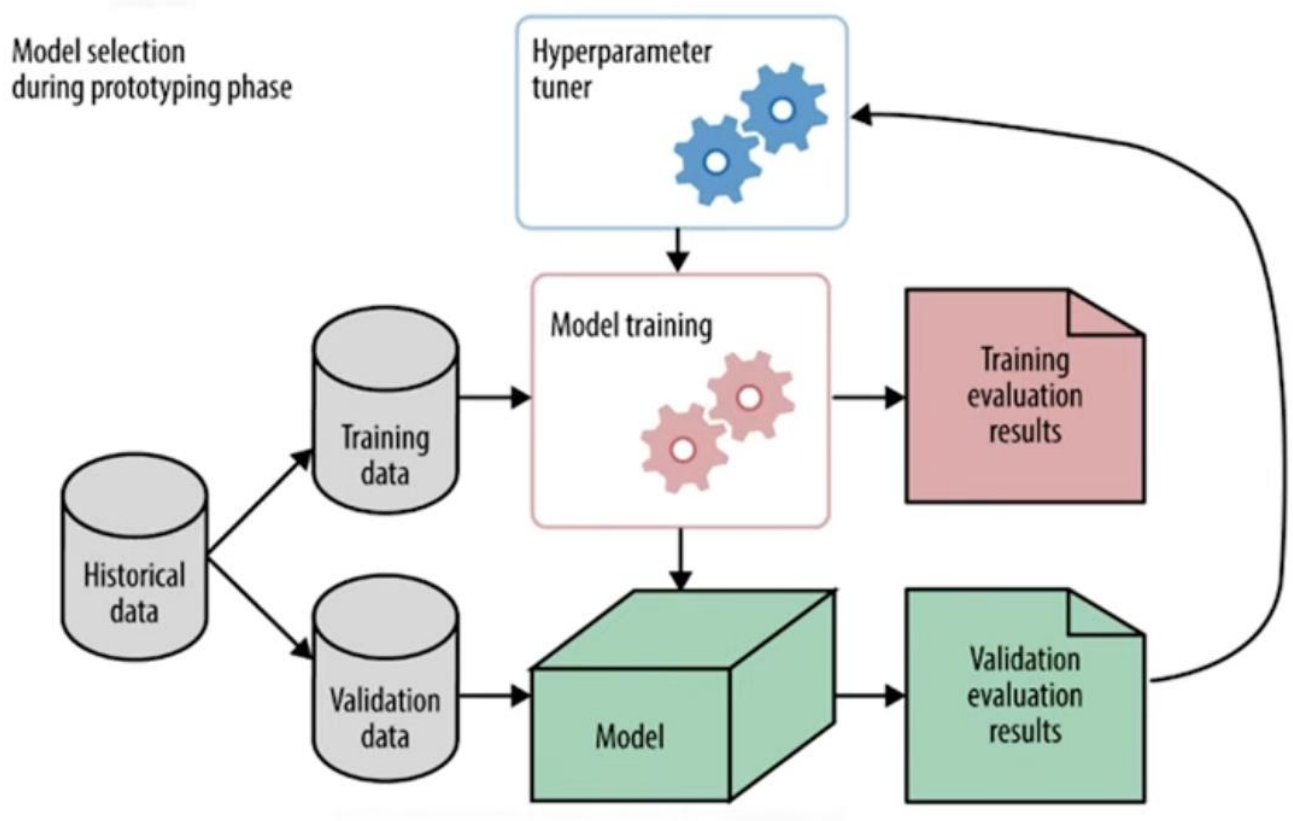
Параметры и гиперпараметры модели

Параметры модели – это значения, которые оцениваются в процессе обучения на данных и отражают знания, извлечённые моделью из обучающей выборки. Например, веса в линейной регрессии, пороги разбиения в решающем дереве. Могут меняться в процессе обучения.

Гиперпараметры – это настройки, задаваемые до начала обучения, которые определяют архитектуру модели и процесс оптимизации. Они влияют на качество и устойчивость модели. Выбираются вручную, подбором или на основе ранее проведенных исследований. Могут быть скорректированы в процессе обучения. Набор гиеперпараметров разный для различных моделей. Например, скорость обучения, глубина дерева решений, коэффициент регуляризации.

Параметры являются результатом обучения, а гиперпараметры – условиями, в которых это обучение осуществляется.

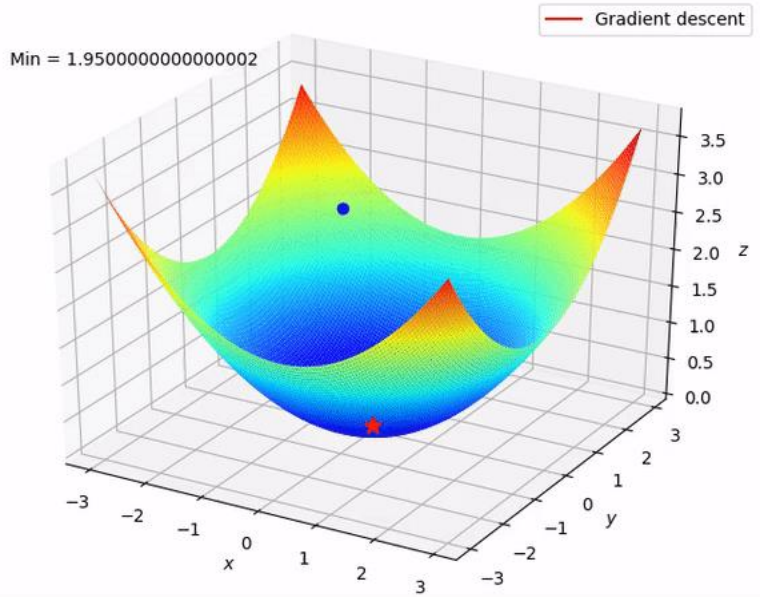
Параметры и гиперпараметры модели



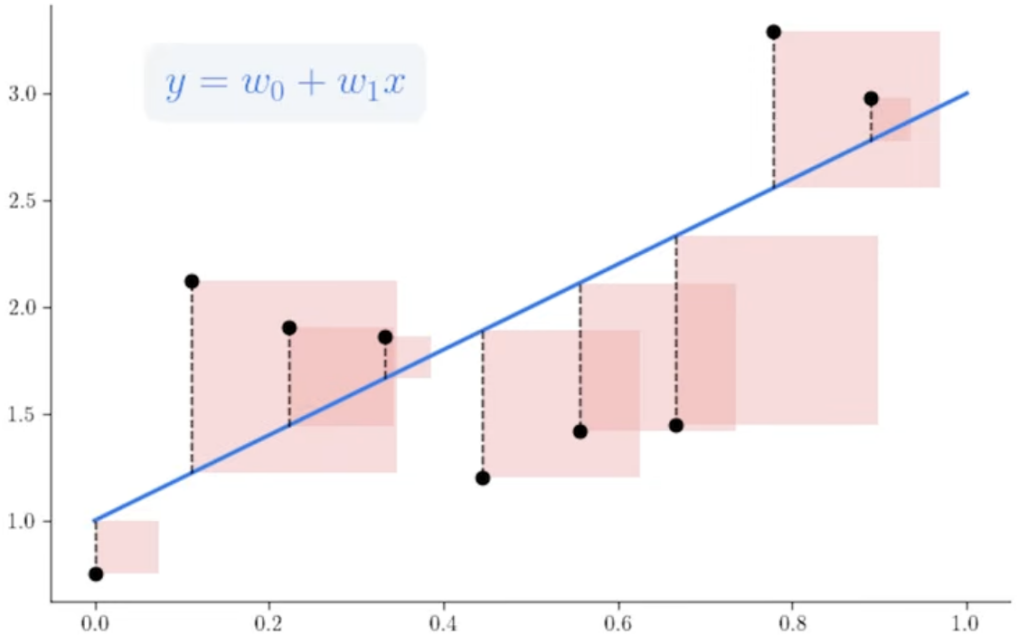
Функция потерь

Функция потерь (Loss Function) – это количественная мера ошибки предсказаний модели, минимизация которой лежит в основе процесса обучения. Может быть не интуитивно понятна для неспециалиста.

- MSE: средняя квадратичная ошибка
- Log Loss: точность предсказанных вероятностей
- Within-Cluster Sum of Squares (WCSS): сумма квадратов внутрикластерных отклонений



Функция потерь для линейной регрессии



$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

N: average over all results

y_i: true y

$\hat{y}_i$: estimate of y

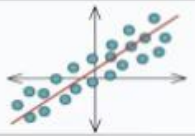
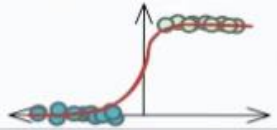
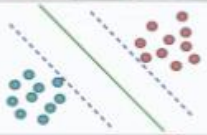
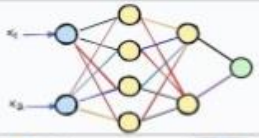
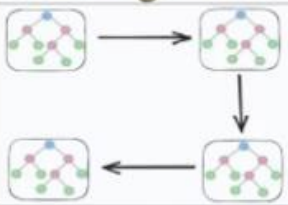
(squaring makes result quadratic)

loss function for linear regression

Функции потерь для разных задач



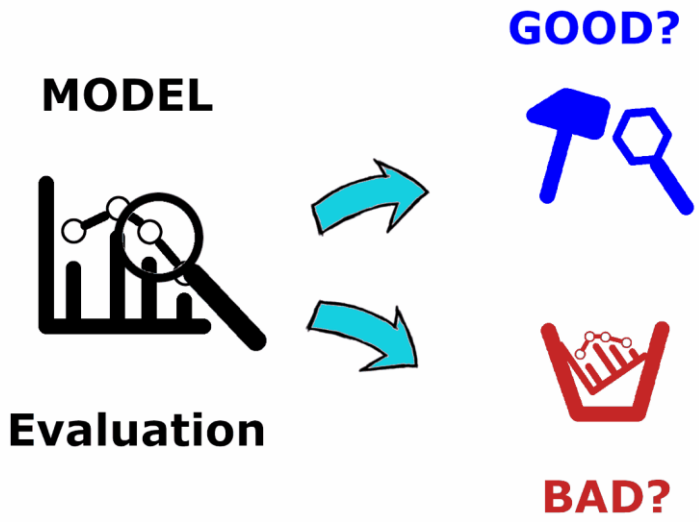
Типичные функции потерь

	Linear Regression	Mean Squared Error
	Logistic Regression	Cross-Entropy Loss
	Decision Tree Classifier	Information Gain or Gini impurity
	Decision Tree Regressor	Mean Squared Error
	Random Forest Classifier	Information Gain or Gini impurity
	Random Forest Regressor	Mean Squared Error
	Support Vector Machines (SVMs)	Hinge Loss
	k-Nearest Neighbors	No loss function
$P(B\|A) = \frac{P(B \cap A)}{P(A)}$	Naive Bayes	No loss function
	Neural Networks	Regression: Mean Squared Error Classification: Cross-Entropy Loss
	AdaBoost	Exponential loss
	Gradient Boosting | LightGBM | CatBoost | XGBoost	Regression: Mean Squared Error Classification: Cross-Entropy Loss

Метрики качества

Метрики качества (Performance Metrics) используются для оценки качества моделей и сравнения различных алгоритмов. Различаются для разных видов задач. Главное их свойство – должны быть понятными (в том числе для неспециалистов) и интерпретируемыми.

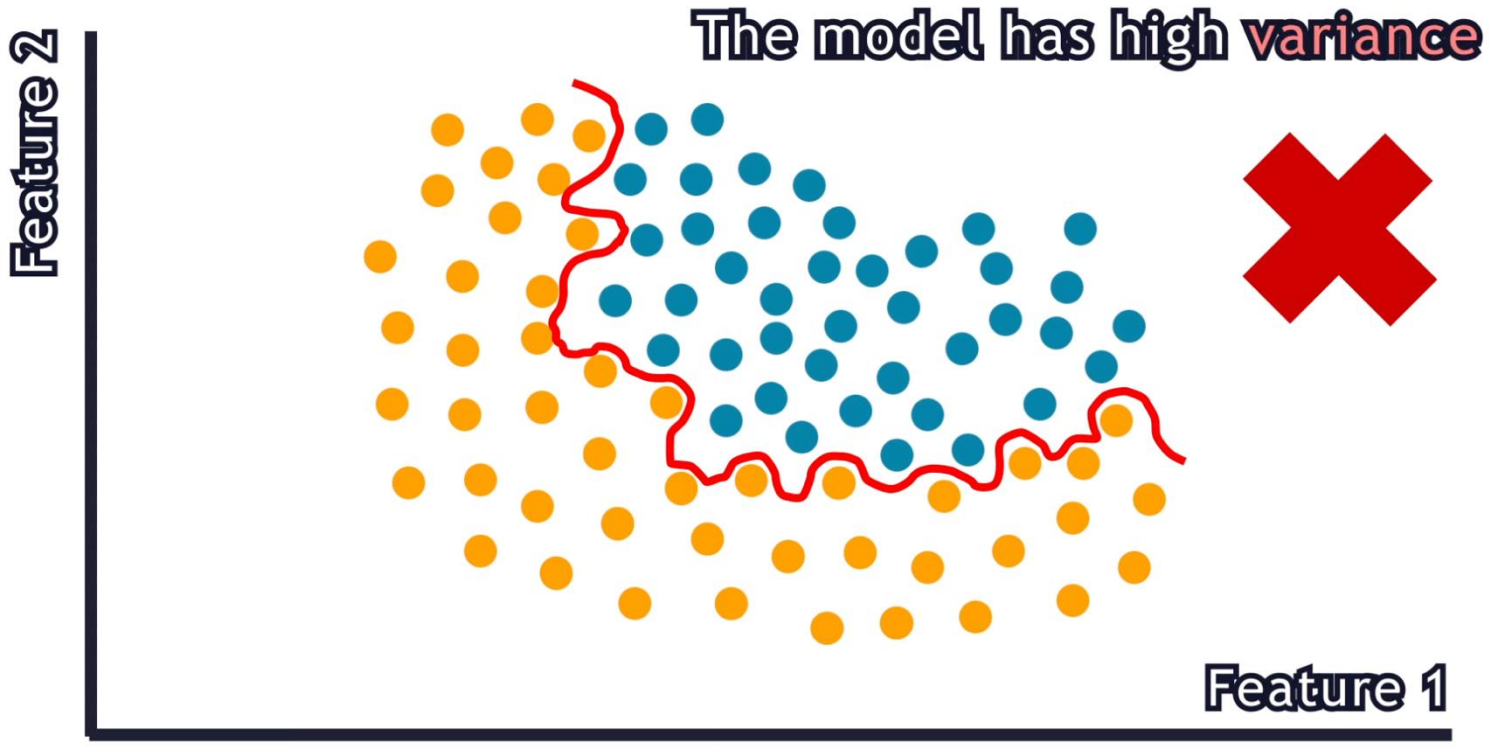
- Accuracy (точность): доля правильных предсказаний
- R^2: коэффициент детерминации (доля объясненной моделью вариации значений зависимой переменной)
- Silhouette Score: оценка качества кластеризации (баланс между компактностью кластеров и их разделённостью)



Проблема переобучения

Переобучение (Overfitting) – это ситуация, когда модель слишком хорошо запоминает обучающие данные, включая шум и случайные особенности, и поэтому плохо работает на новых данных. Такая модель не обладает обобщающей способностью. Причины могут быть разные: слишком сложная модель, малый объём данных, шумные данные, слишком много признаков и прочее. Обычно в этом случае метрики качества на тренировочной выборке показывают хорошие результаты, а на тестовой – значительно хуже. Есть разные способы борьбы с этим: регуляризация, упрощение модели, расширение выборки, ансамблевые модели.

Переобучение



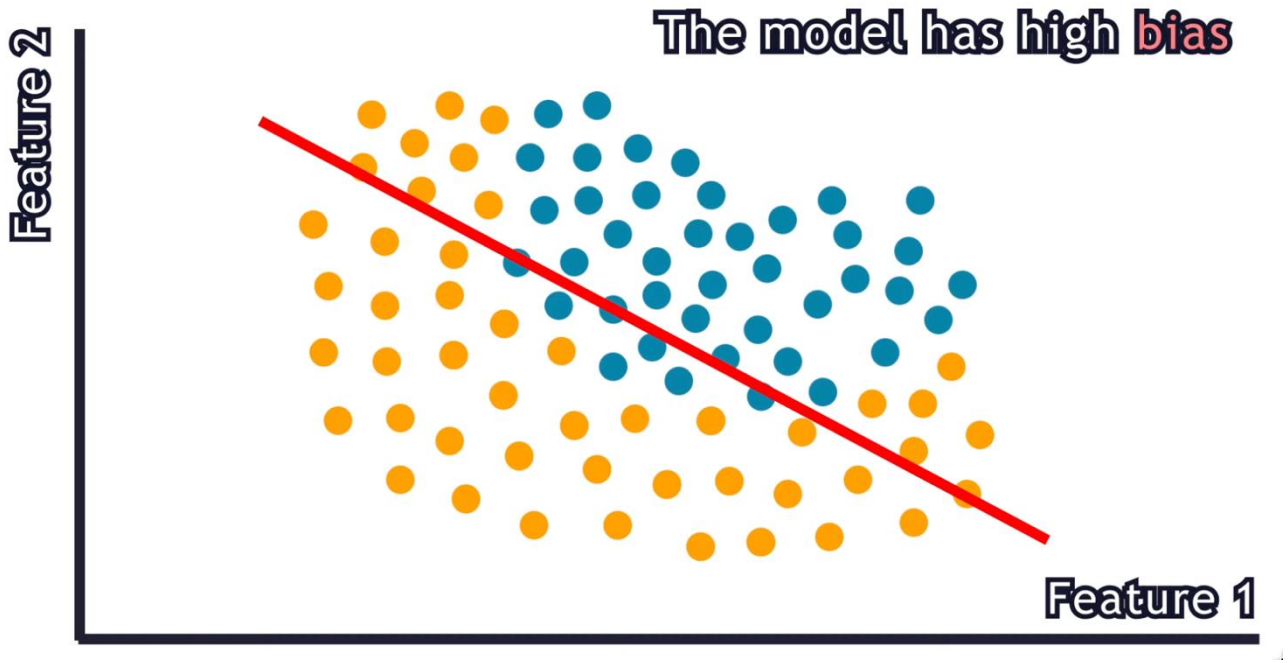
Проблема недообучения

Проблема недообучения (underfitting) - модель машинного обучения слишком простая или не подходит, чтобы выявить важные закономерности в данных, что приводит к плохим метрикам качества как на обучающих, так и на тестовых данных.

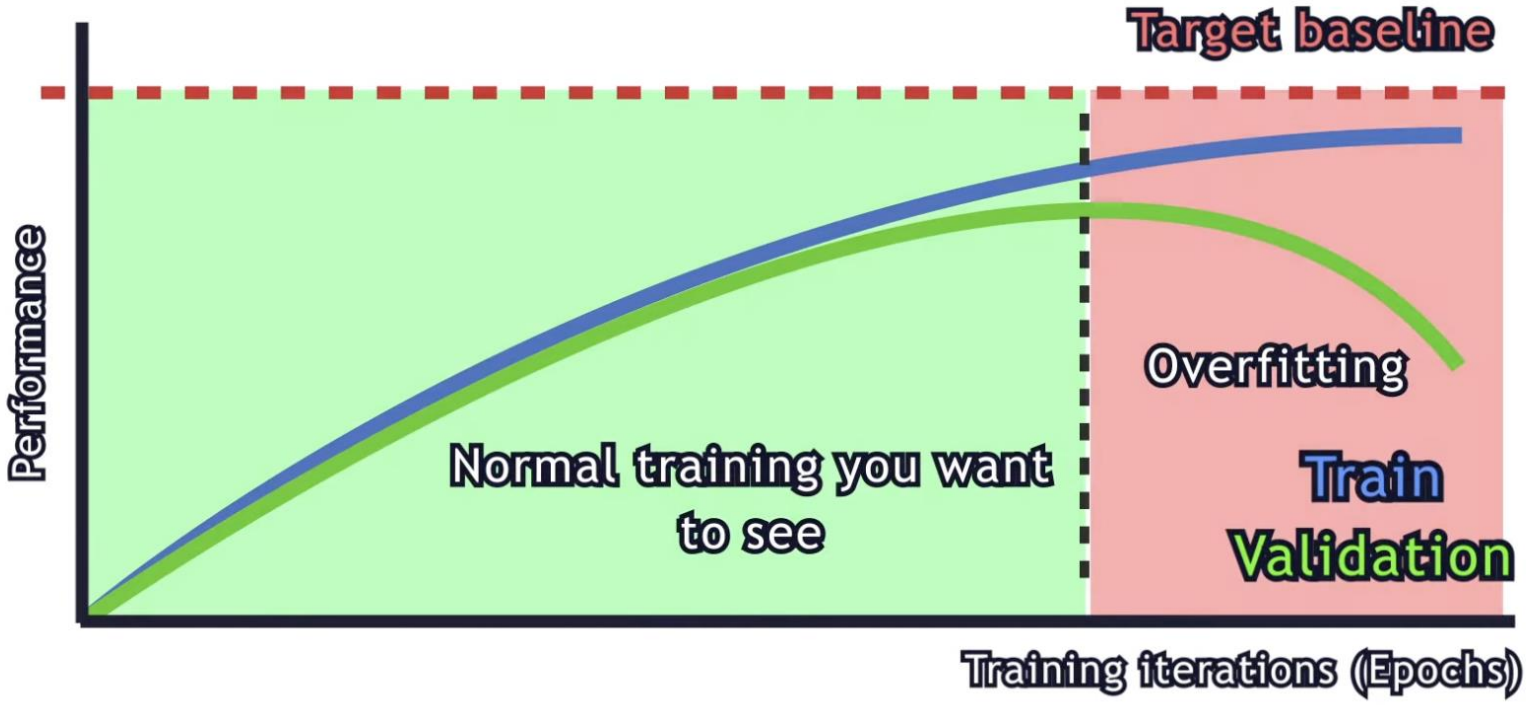
Возможные решения:

- использовать более сложный алгоритм (например, вместо линейной регрессии случайный лес);
- устранить ошибки в данных и улучшить их предобработку;
- улучшить признаки.

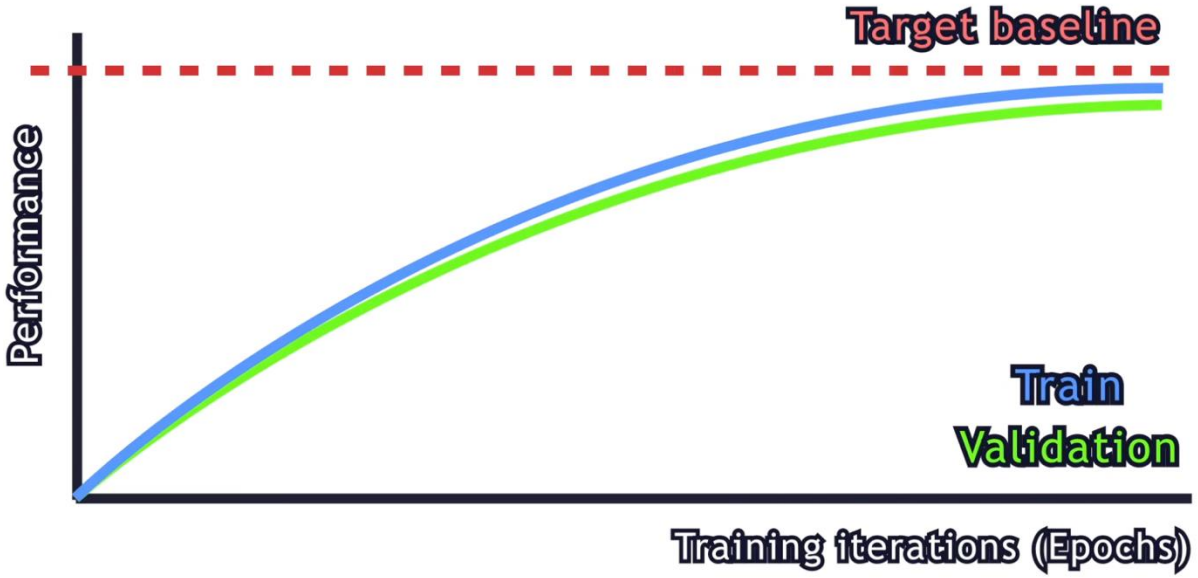
Недообучение

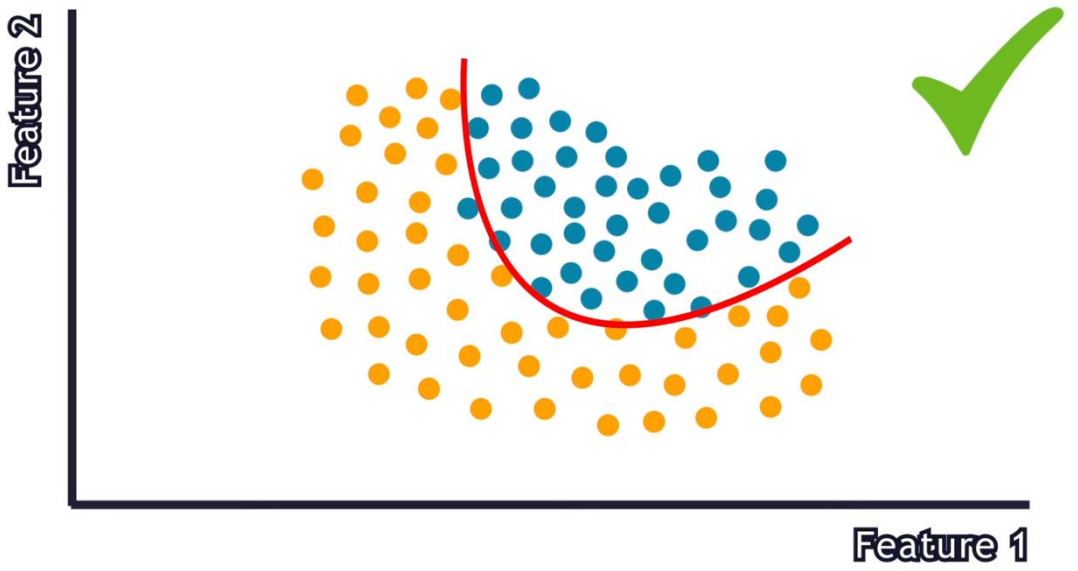


Переобучение

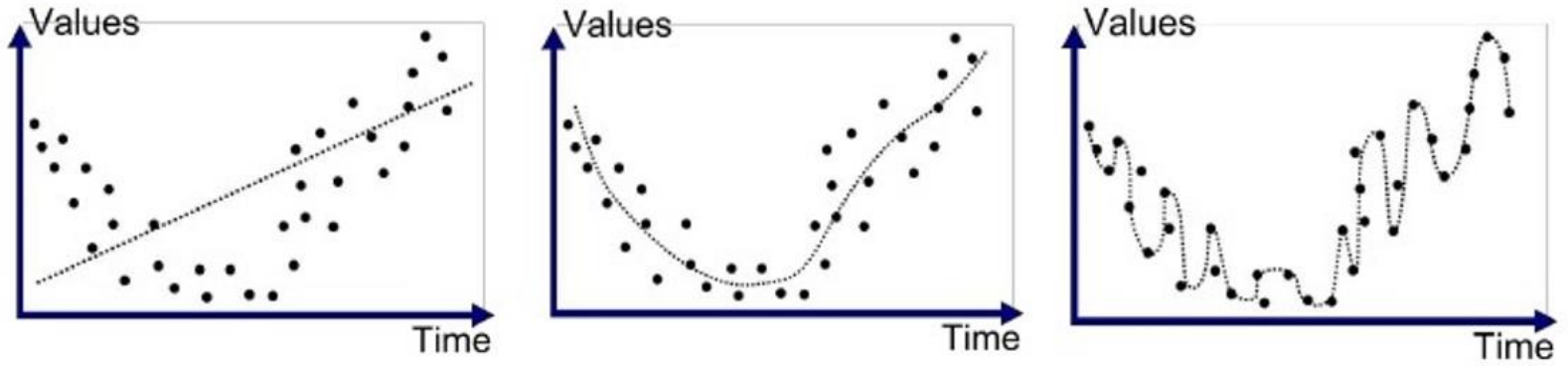


Хорошая модель





Переобучение, недообучение и хорошая модель



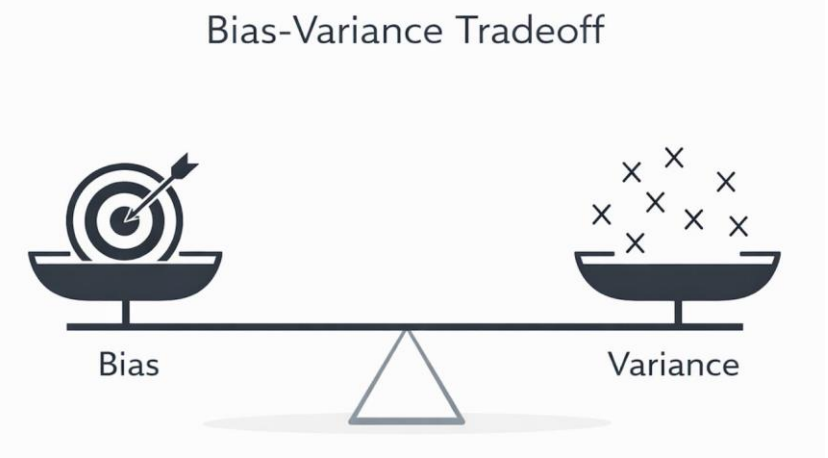
Underfitted | Good Fit/Robust | Overfitted

Компромисс между смещением и разбросом модели

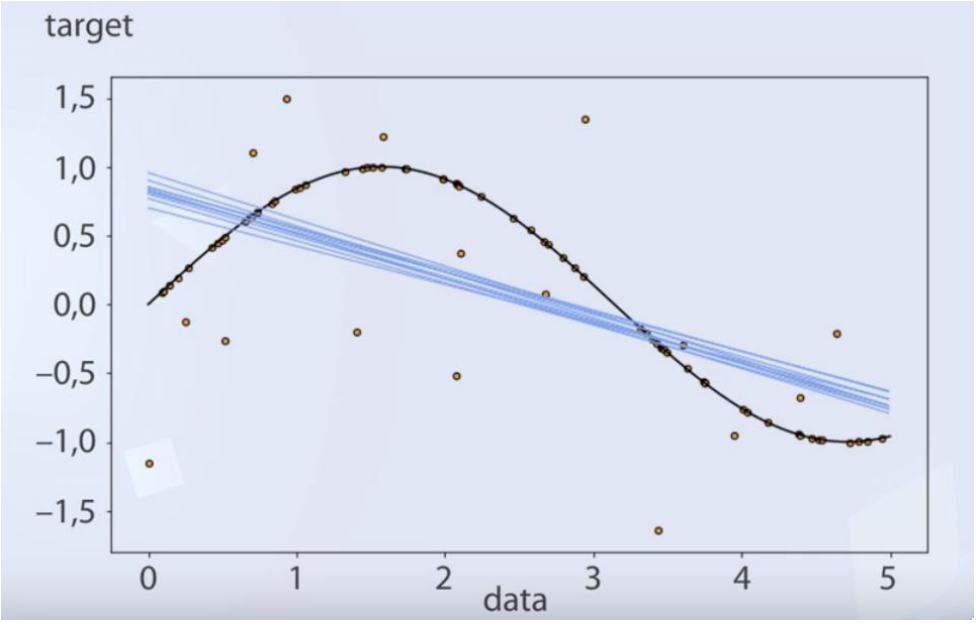
Ошибка модели складывается из трех компонент:

- **Разброс (variance)** – устойчивость модели к изменениям в обучающей выборке.
- **Смещение (bias)** – модель не может уловить истинную зависимость.
- **Шум (noise)** – характеристика сложности и противоречивости данных

Bias-variance tradeoff описывает компромисс между сложностью модели и её способностью обобщать.

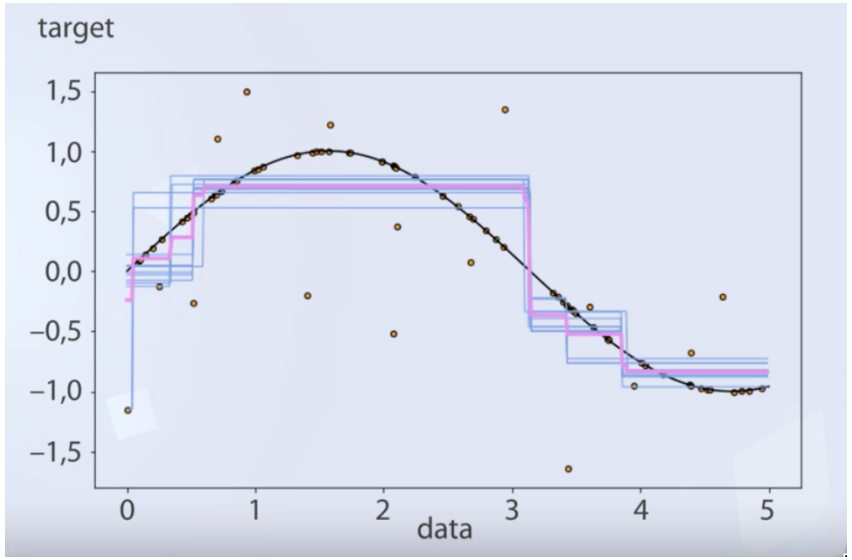


Низкий разброс и высокое смещение



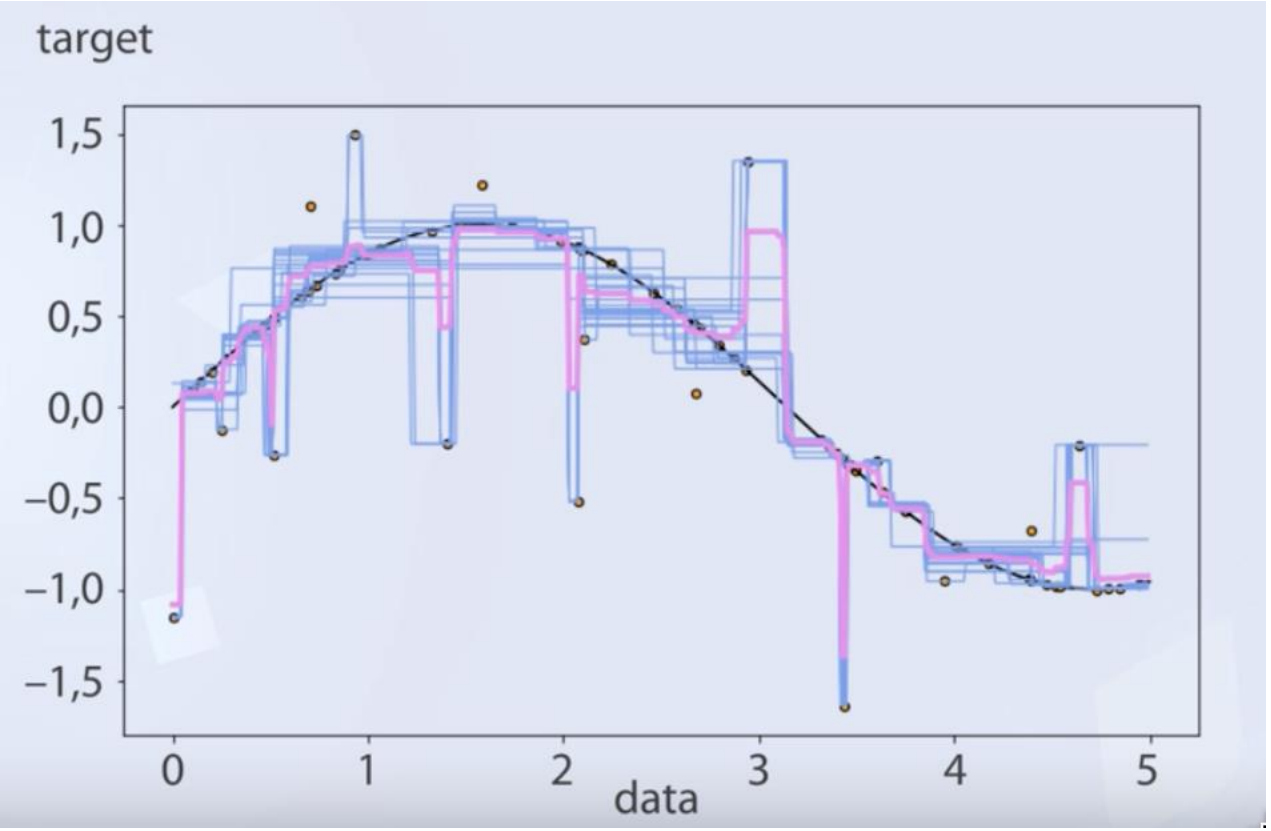
Нелинейная модель хорошо подходит. Линейная модель – плохо. При этом, если обучать линейную модель на разных подвыборках объектов, то мы получим несколько синих линий, которые очень близки друг к другу. Следовательно в этом примере линейная модель характеризуется низким разбросом, модель очень устойчива. А смещение у нее высокое, она очень плохо аппроксимирует данные.

Невысокий разброс и невысокое смещение



Модели решающего дерева лучше аппроксимируют, чем линейная регрессия и разброс между ними не очень высокий, то есть они не сильно меняются при построении на разных подвыборках.

Высокий разброс и низкое смещение

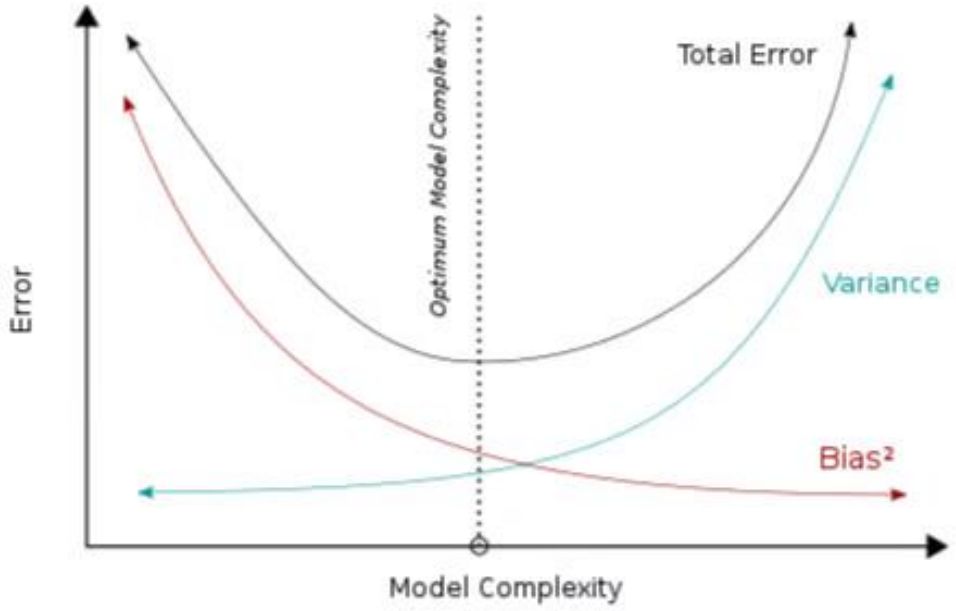


Модели еще лучше аппроксимируют, но разброс между ними становится выше. Это происходит при росте глубины решающего дерева, появляется эффект переобучения.

Компромисс между смещением и разбросом

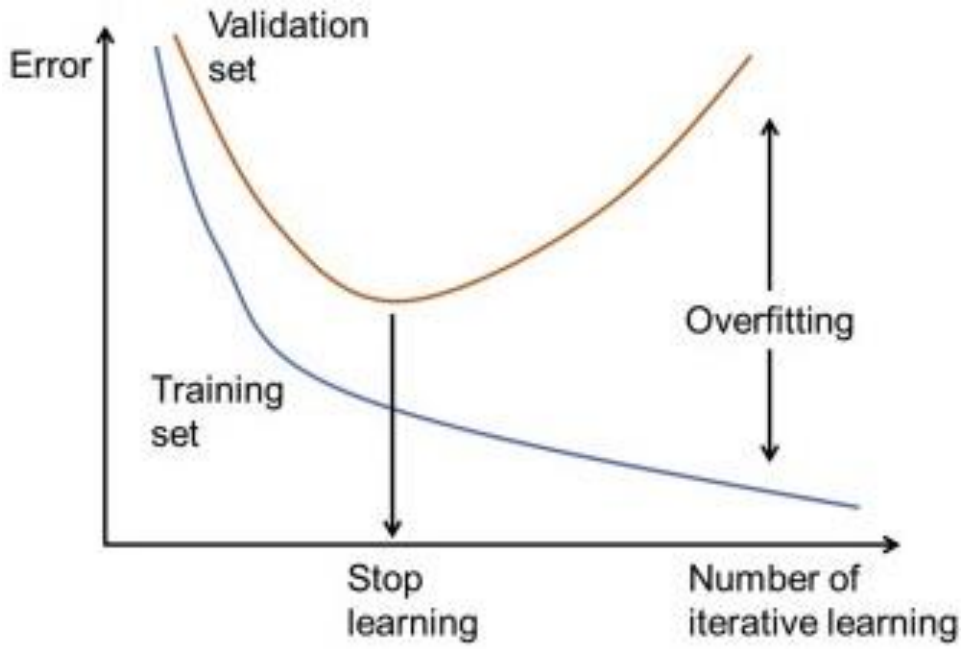
Усложняем модель → bias уменьшается, variance растёт.
Упрощаем модель → bias растёт, variance уменьшается.
Цель компромисса - найти правильный баланс, который минимизирует общую ошибку. Для этого можно использовать разные стратегии:

- Выбрать модель, соответствующую сложности данных.
- Использовать ансамблевые методы, чтобы объединить прогнозы нескольких моделей.
- Увеличить размер и разнообразие обучающих данных и пр.



Кривые обучения

Кривые обучения (Learning Curves) показывают, как качество модели меняется с ростом объёма данных или итераций обучения. Их используют, чтобы понять, что именно мешает модели учиться и что делать дальше.



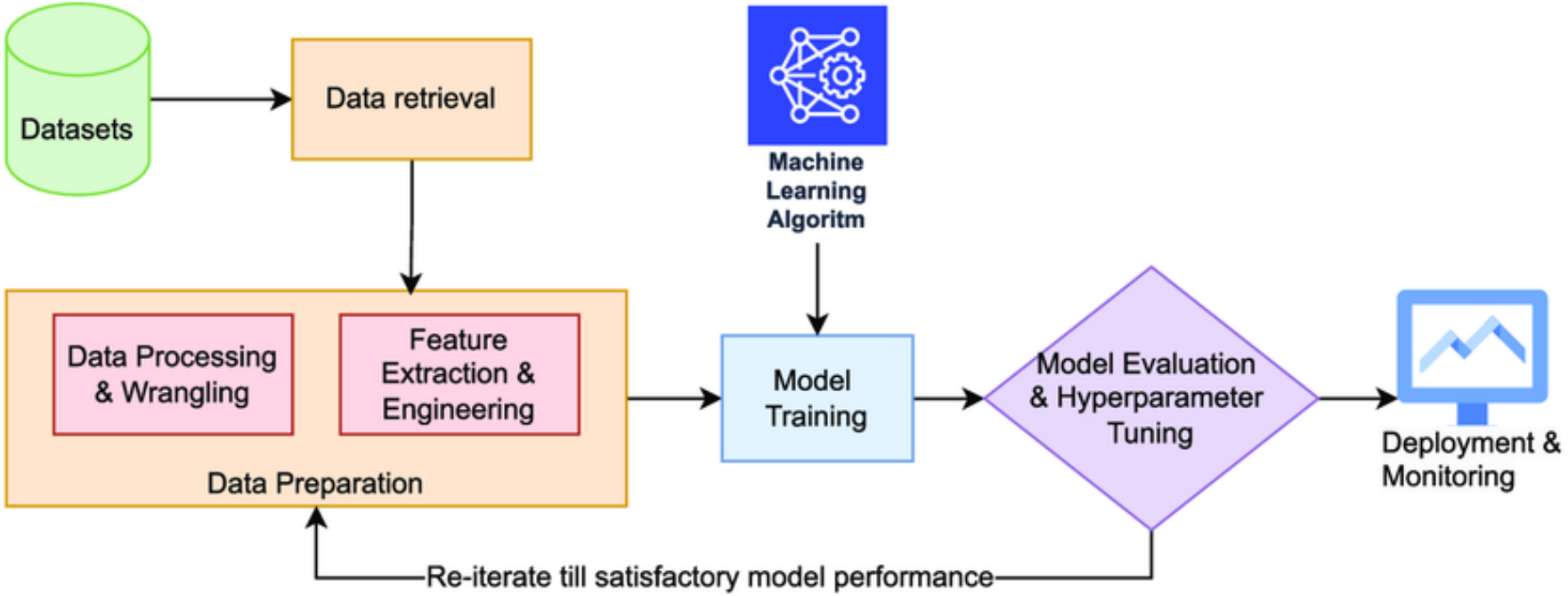
Применение модели

После того как модель обучена она применяется для предсказания значений целевой переменной на новых данных.

В случае с задачей скоринга на основе данных о клиентах (которые отсутствовали в обучающей выборке) мы предсказываем по каждому новому клиенту вернет ли он кредит в срок или нет.

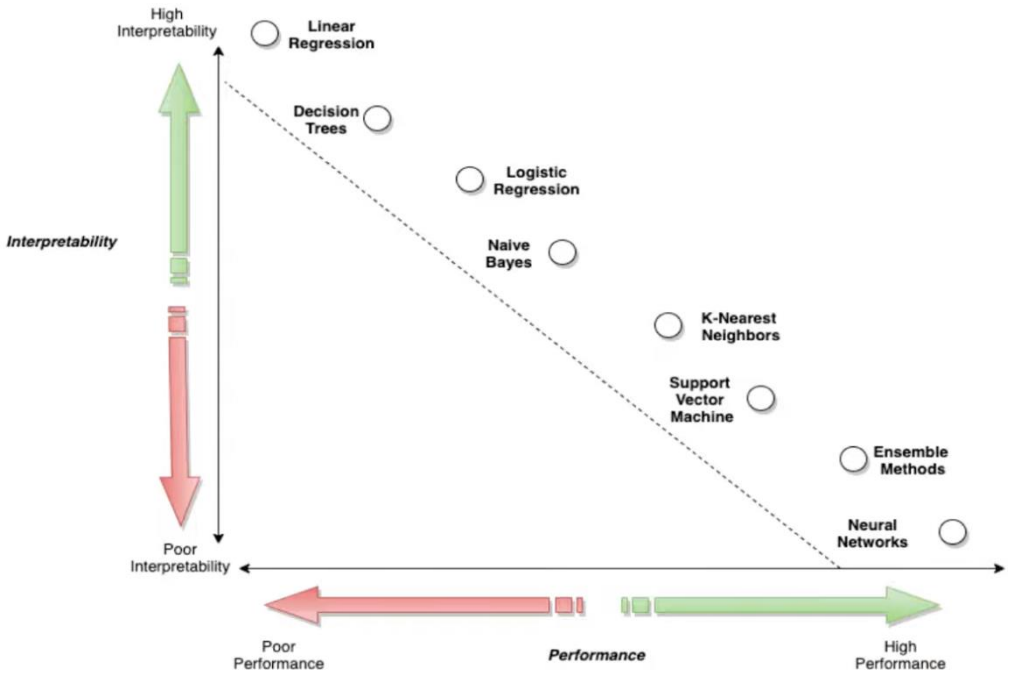
ML-pipeline

ML-pipeline – это последовательность этапов работы с данными, обучения и оценки модели, оформленная как единый воспроизводимый процесс.



Рекомендации

Начинайте с простых моделей, постепенно усложняя. Учитывайте, что сложные модели требуют больше данных и вычислительных мощностей. Дольше обучаются. Их сложнее интерпретировать и отлаживать, а индивидуальное влияние каждого предиктора в ряде случаев невозможно оценить. Более затруднительная дальнейшая эксплуатация (внедрение и обслуживание).

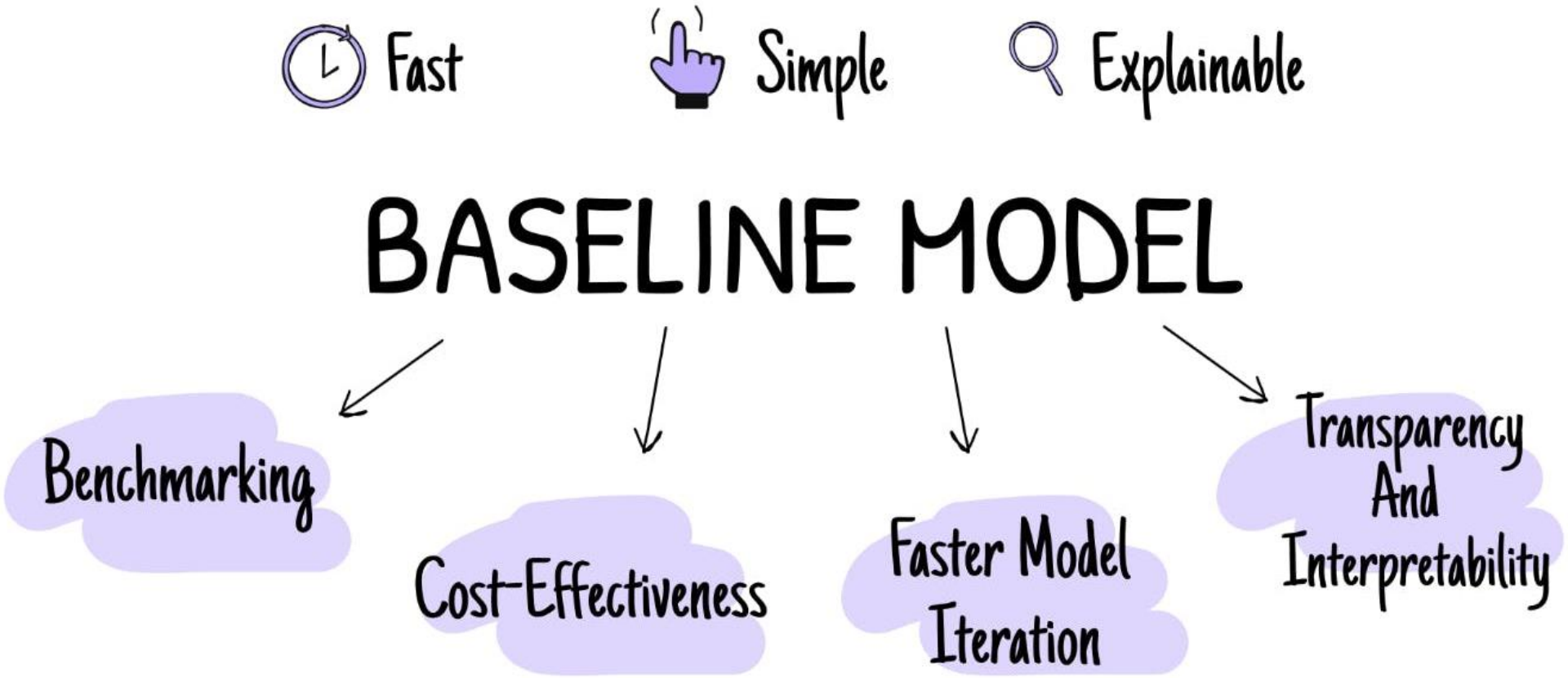


Базовая модель

Базовая модель (Baseline Model) в машинном обучении используют как точку отсчёта и минимальный ориентир качества, чтобы понять, насколько более сложная модель лучше простых решений. Если модель не лучше базовой, то она бесполезна.

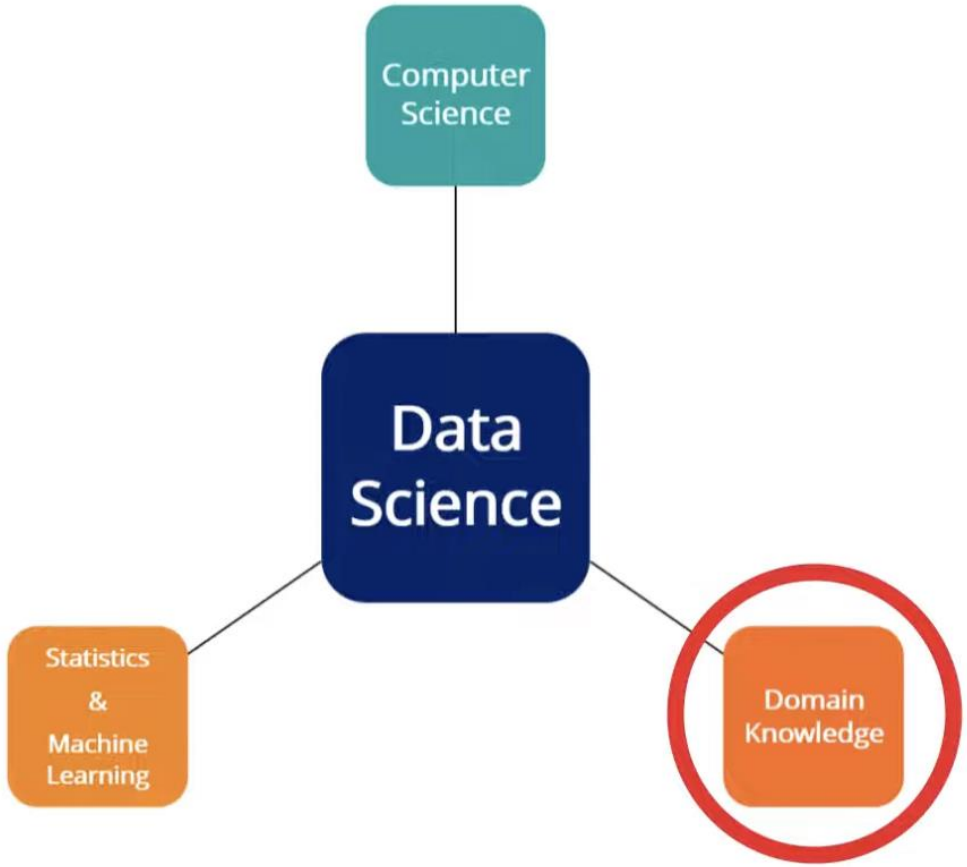
В определенных ситуациях базовой модели оказывается достаточно, учитывая, что она проще, стабильнее и дешевле в поддержке. Простые модели более понятны для бизнеса и команды, легче интерпретируемы. По мере роста сложности модели должна расти ее производительность.

Качественное конструирование признаков и глубокое погружение в проблему часто важнее, чем сложная модель.


Fast
Simple
Explainable
BASELINE MODEL
Benchmarking
Cost-Effectiveness
Faster Model Iteration
Transparency And Interpretability

Понимание предметной области

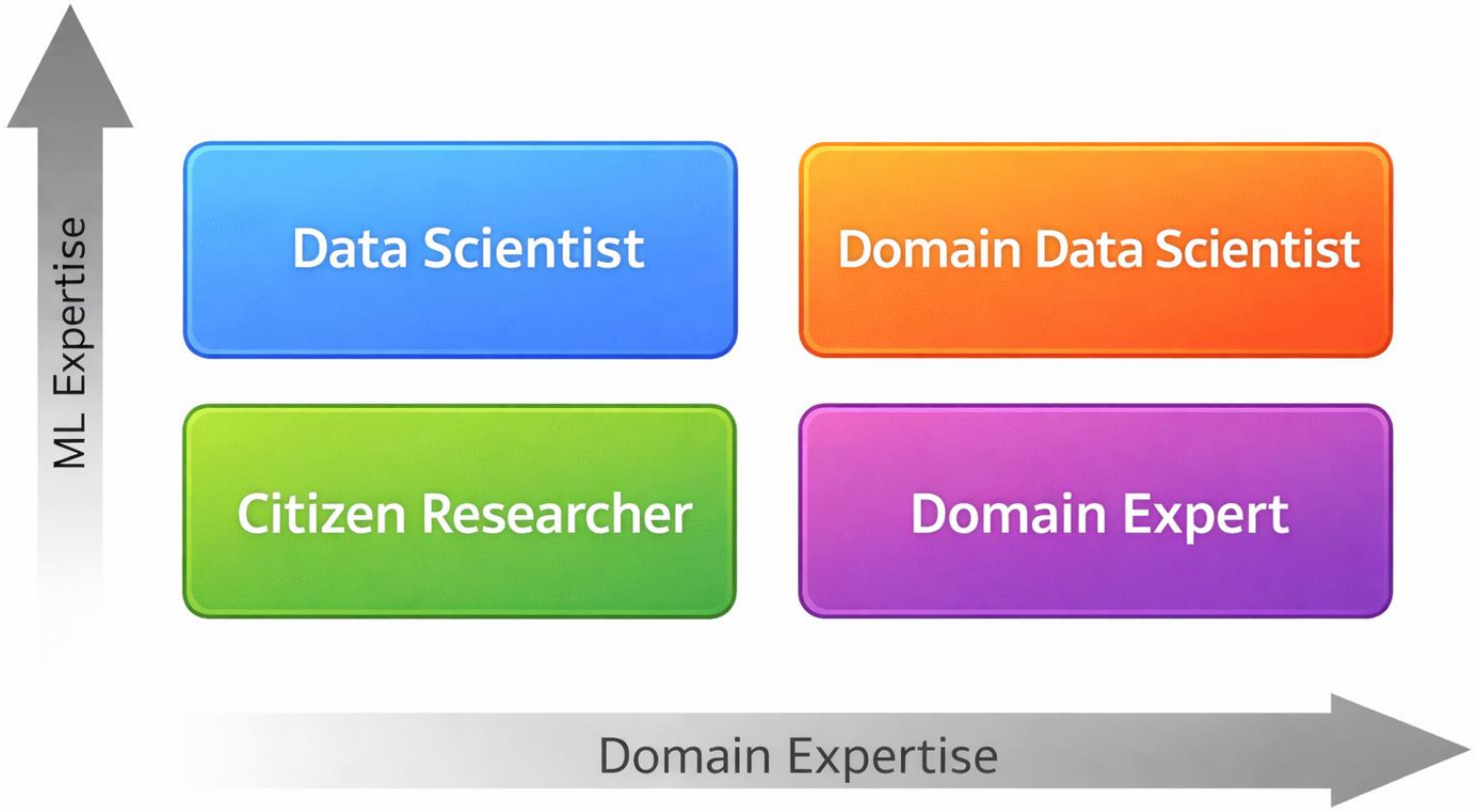
Понимание предметной области связывает математическую модель с реальным миром. Без знания контекста можно построить модель с хорошими метриками, которая при этом решает не ту задачу или делает бессмысленные с практической точки зрения предсказания. Глубокое понимание предметной области помогает правильно сформулировать цель, выбрать релевантные признаки, интерпретировать данные, выявить ошибки и ограничения, а также оценить, насколько результаты модели применимы на практике. В итоге именно предметная экспертиза превращает ML-алгоритм из абстрактного инструмента в рабочее и полезное решение.



Привлечение доменных экспертов

Доменные эксперты привносят глубокое понимание реальных процессов, стоящих за данными. Они помогают выделять действительно важные признаки, своевременно выявлять проблемы с качеством данных и оценивать, насколько предсказания модели имеют смысл в реальном мире. Без участия доменных экспертов команда рискует оптимизировать модель под неподходящие метрики или упустить критически важные ограничения задачи – бизнес-правила, физические пределы, юридические или практические требования. Взаимодействие с доменными экспертами позволяет сделать ML-решение осмысленным, надёжным и применимым на практике.

Доменная экспертиза



Дизайн-документ



Дизайн-документ (Design Doc) – это подробное описание того, что именно планируется реализовать в рамках проекта. В нем нет кода, конкретных результатов расчетов или значений метрик. Основная цель диздока – до начала реализации проекта описать и согласовать его ключевые элементы со всей командой. На его основе можно предварительно обсудить проект, согласовать его, и даже обнаружить проблемы и несостыковки до начала активной фазы реализации.

Содержание: бизнес-цель, описание данных, подход к моделированию, процесс обучения и метрики, архитектура, деплой и мониторинг, риски.

Инструменты



Язык программирования Python 3

- Дистрибутив Anaconda содержит интерпретатор языка Python и несколько сред разработки. Мы будем использовать интерактивную среду разработки Jupyter Notebook, работающую с Python



  https://www.anaconda.com/

- Google Colab (интерактивная облачная среда для работы с кодом)

  https://colab.research.google.com/



Jupyter Notebook



Jupyter Notebook – это среда разработки, со следующим особенностями:

- можно сразу увидеть результат выполнения всего кода или отдельных его фрагментов, при этом код можно разбить на куски и выполнять их в произвольном порядке;
- предусмотрен вывод результата сразу после фрагмента кода;
- поля для ввода кода чередуются с полями, в которые можно добавлять текс, ссылки, изображения.

Спасибо за внимание!